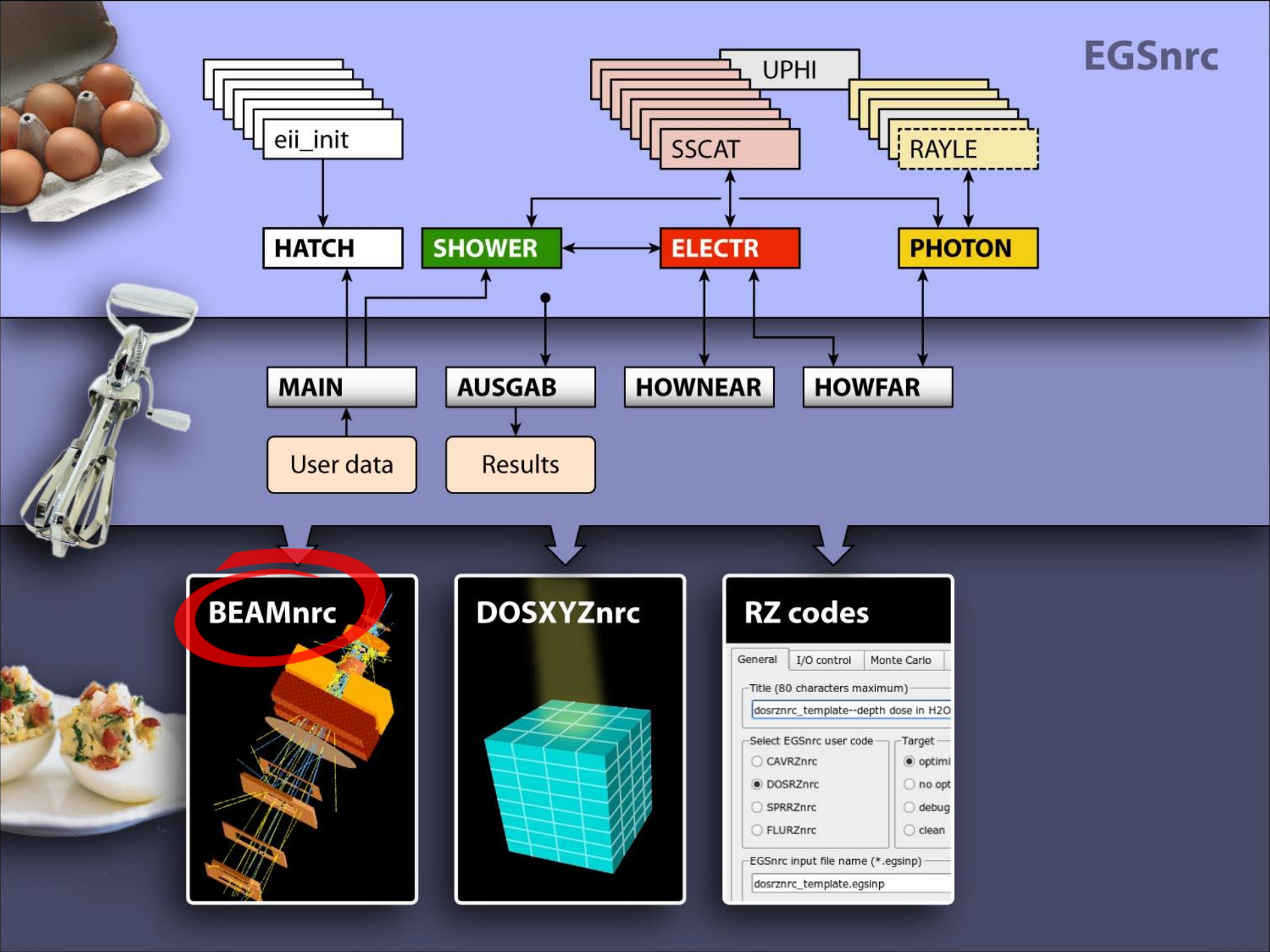
ICTP-SAIFR EGSnrc Monte Carlo School for Medical Physics
IFT-UNESP, São Paulo, Brazil
March 30 – April 3, 2026

LECTURE 14 · EGSnrc course

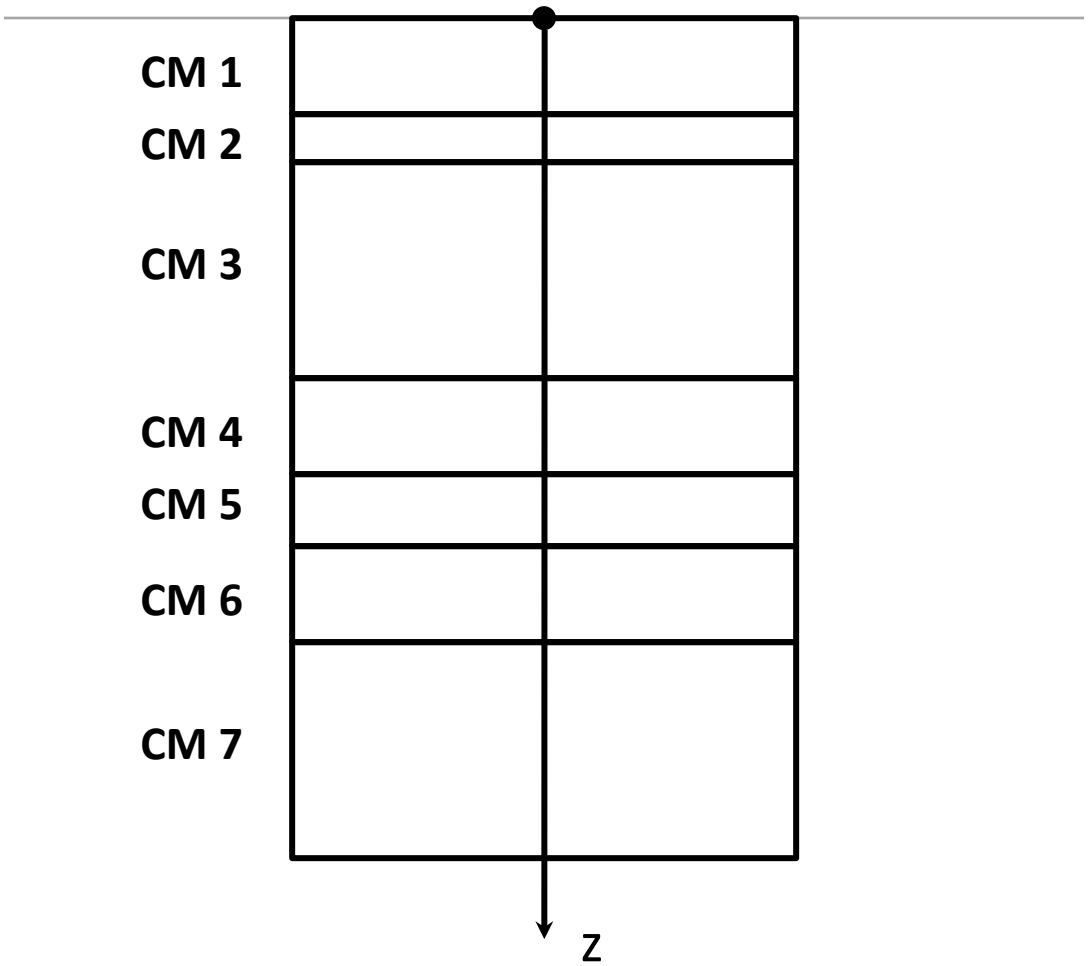
BEAMnrc introduction and main inputs

Frédéric Tessier · National Research Council Canada, Metrology Research Centre

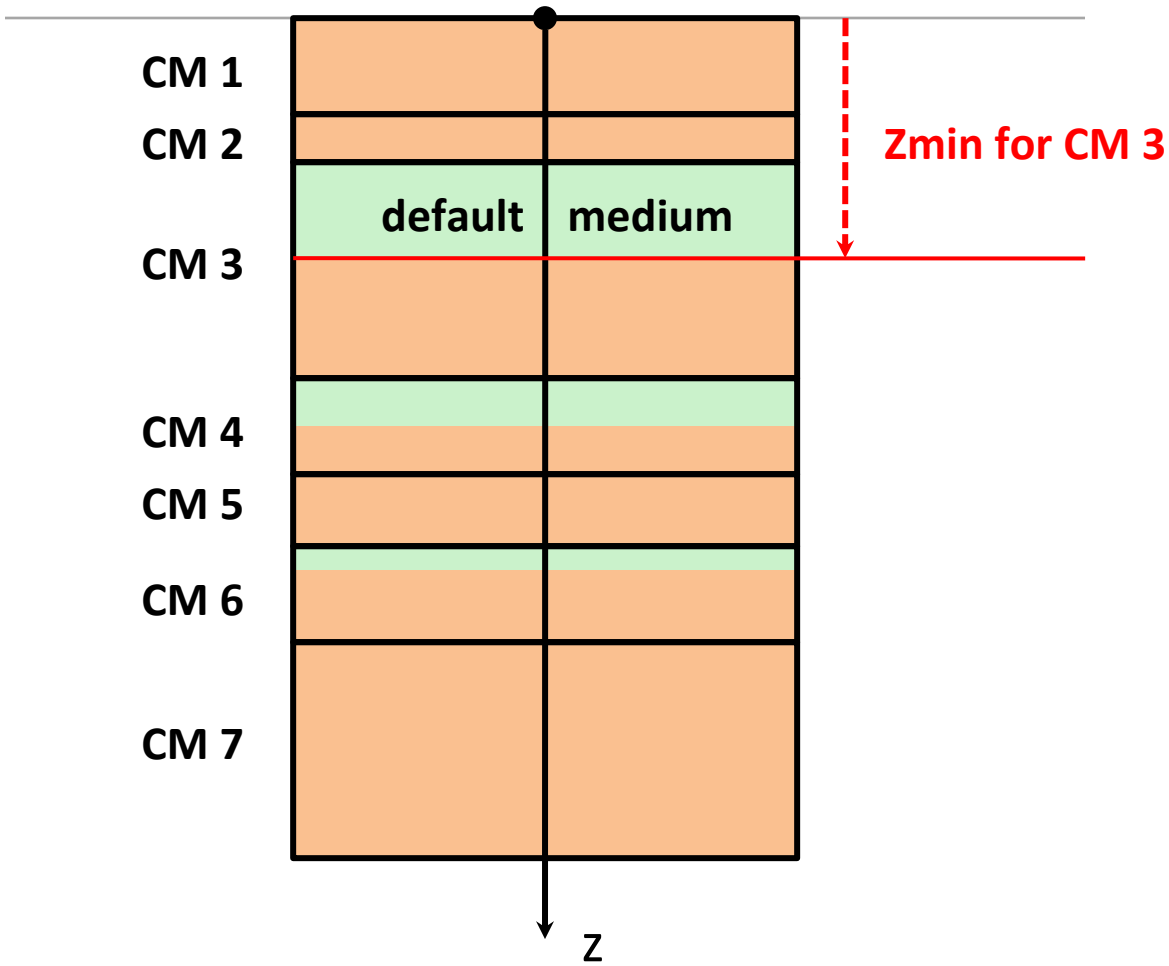
EGSnrc


eii_init
UPHI
SSCAT
RAYLE
HATCH
SHOWER
ELECTR
PHOTON
MAIN
AUSGAB
HOWNEAR
HOWFAR
User data
Results
BEAMnrc
DOSXYZnrc
RZ codes
General
I/O control
Monte Carlo
Title (80 characters maximum)
dosrznrc_template--depth dose in H2O
Select EGSnrc user code
CAVRZnrc
DOSRZnrc
SPRRZnrc
FLURZnrc
Target
optimi
no opt
debug
clean
EGSnrc input file name (*.egsinp)
dosrznrc_template.egsinp

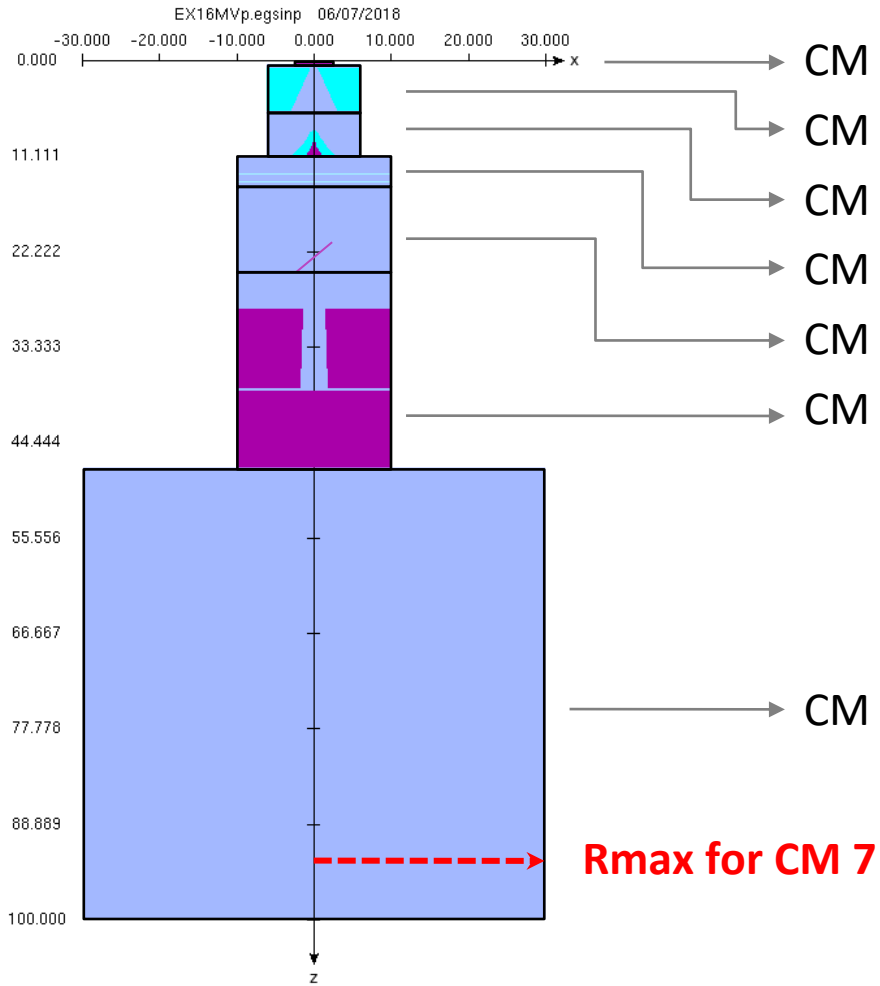
BEAMnrc models a stack of component modules (CM)



BEAMnrc models a stack of component modules (CM)



Each component module has a type and a name



CM 1: **SLABS** "target"

CM 2: **CONS3R** "primcoll"

CM 3: **FLATFILT** "flattenf"

CM 4: **CHAMBER** "chamber"

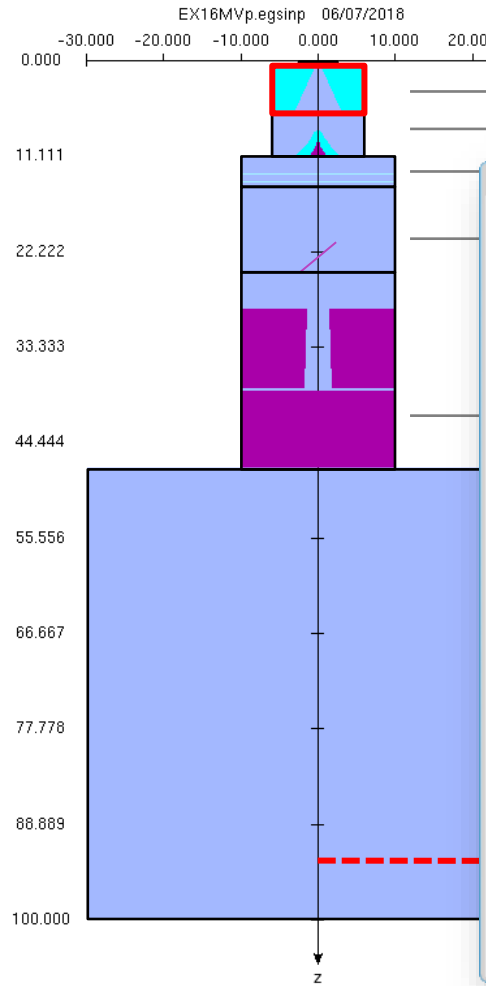
CM 5: **MIRROR** "mirror"

CM 6: **JAWS** "secjaws"

CM 7: **SLABS** "airslab"

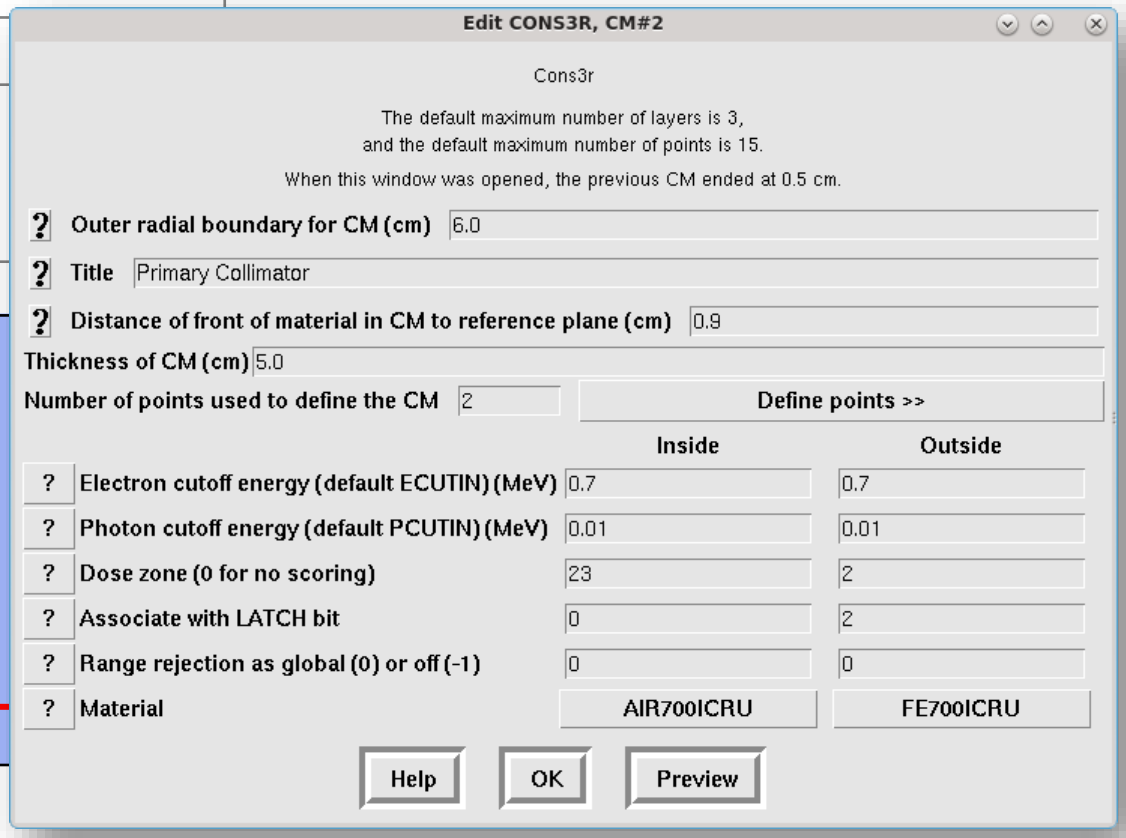
Rmax for CM 7

Edit CM parameters with beamnrc_gui

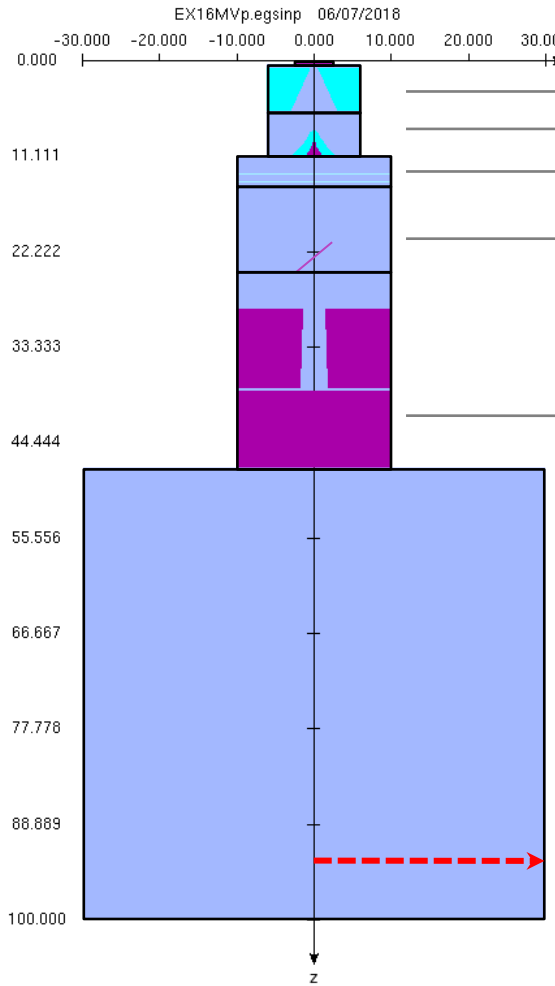


CM 1: **SLABS** "target"

CM 2: **CONS3R** "primcoll"



Edit CM parameters with beamnrc_gui



CM 1: **SLABS** "target"

CM 2: **CONS3R** "primcoll"

CM 3: **FLATFILT** "flattenf"

CM 4: **CHAMBER** "chamber"

CM 5: **MIRROR** "mirror"

CM 6: **JAWS** "secjaws"

CM 7: **SLABS** "airslab"

Rmax for CM 7

Define the accelerator in a `.module` text file

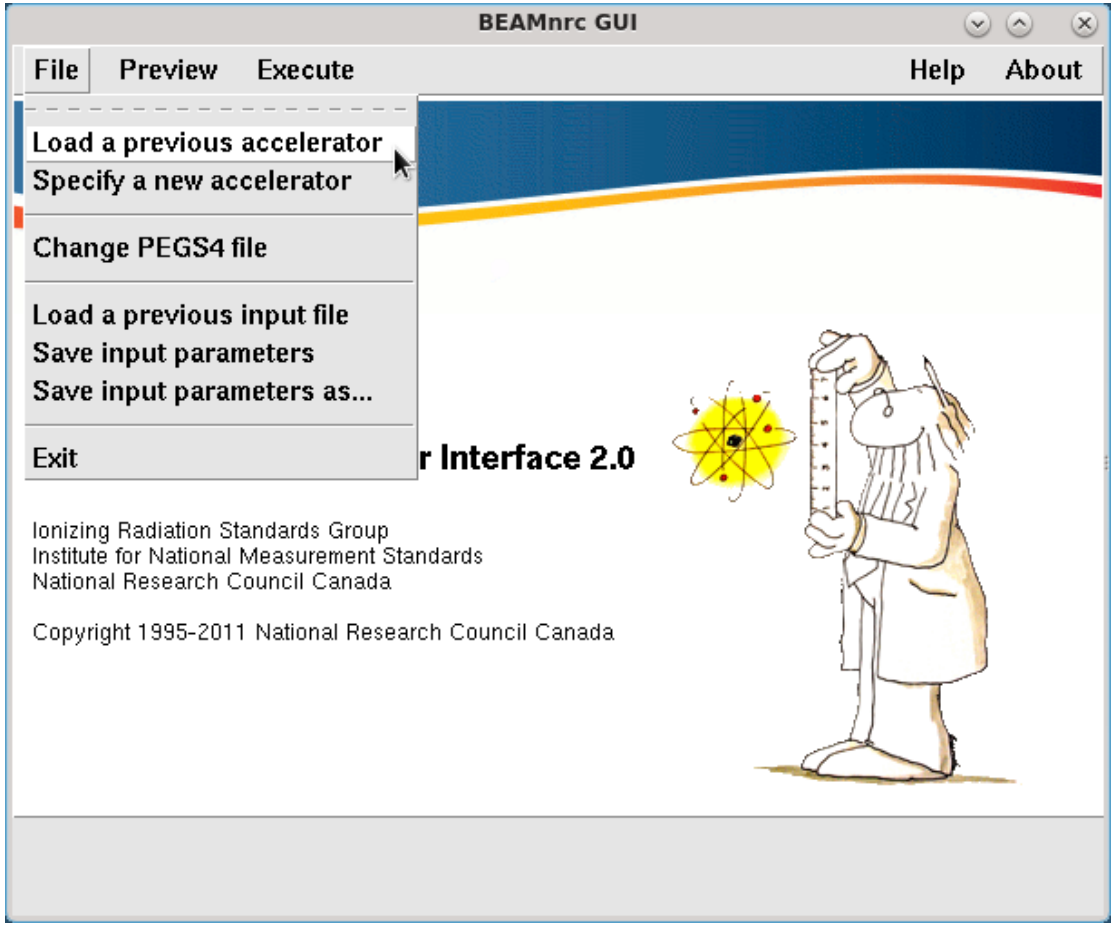
1. **Create text file** `$EGS_HOME/beamnrc/spec_modules/mylinac.module`
   This file specifies what component modules you want inside your accelerator, and their names. For example:

```
CM names:    SLABS CONS3R JAWS SLABS
Identifiers: mytarget mycollim myjaws myair
```

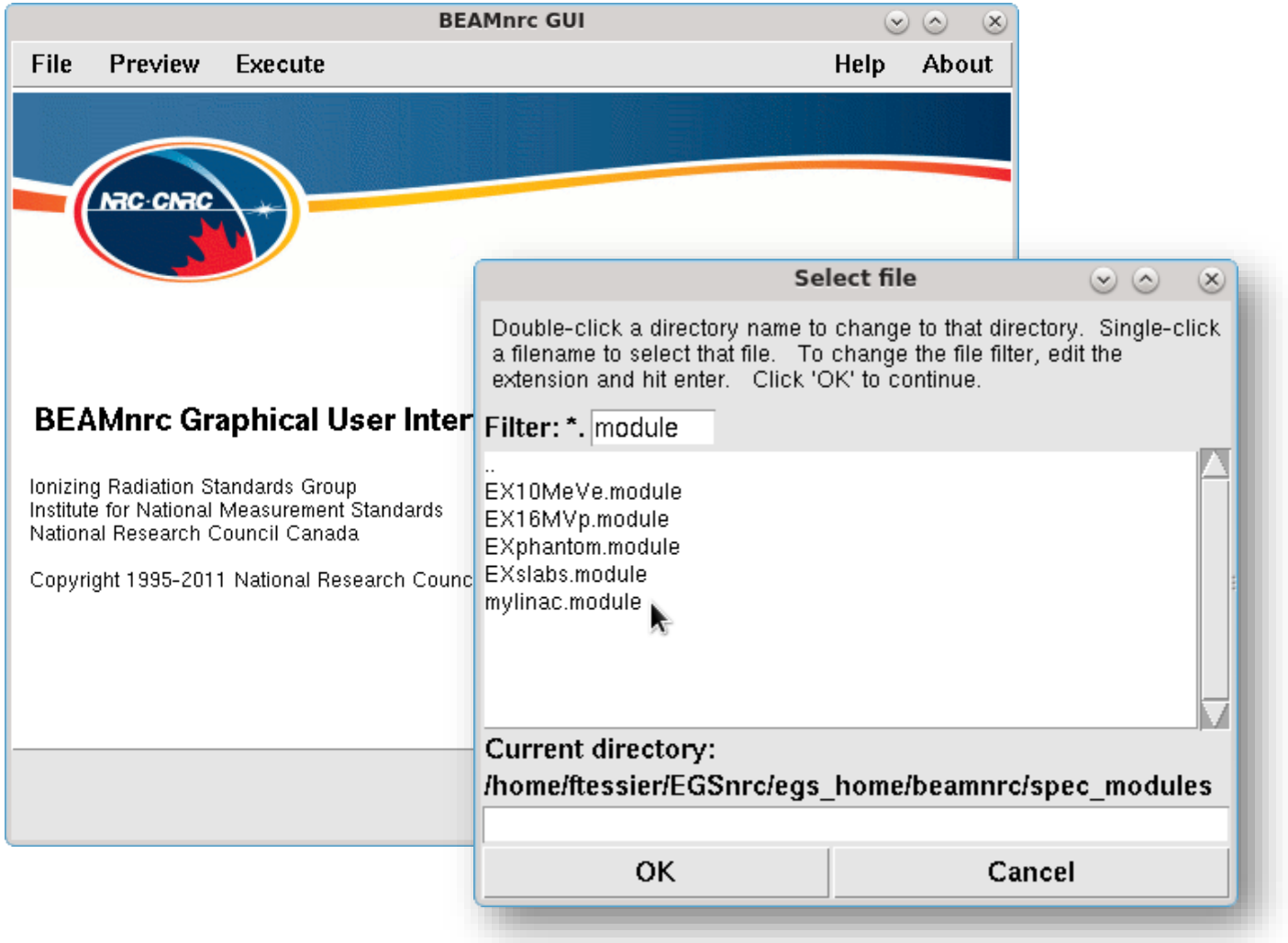
2. **Run** `beam_build.exe mylinac` (no extension)
   This program collects the source code for your accelerator inside a new application directory `$EGS_HOME/BEAM_mylinac/`

3. **Launch** `beamnrc_gui` **and load your accelerator**
   This program is a Tcl/Tk graphical user interface to visualize an accelerator and edit all the simulation parameters.

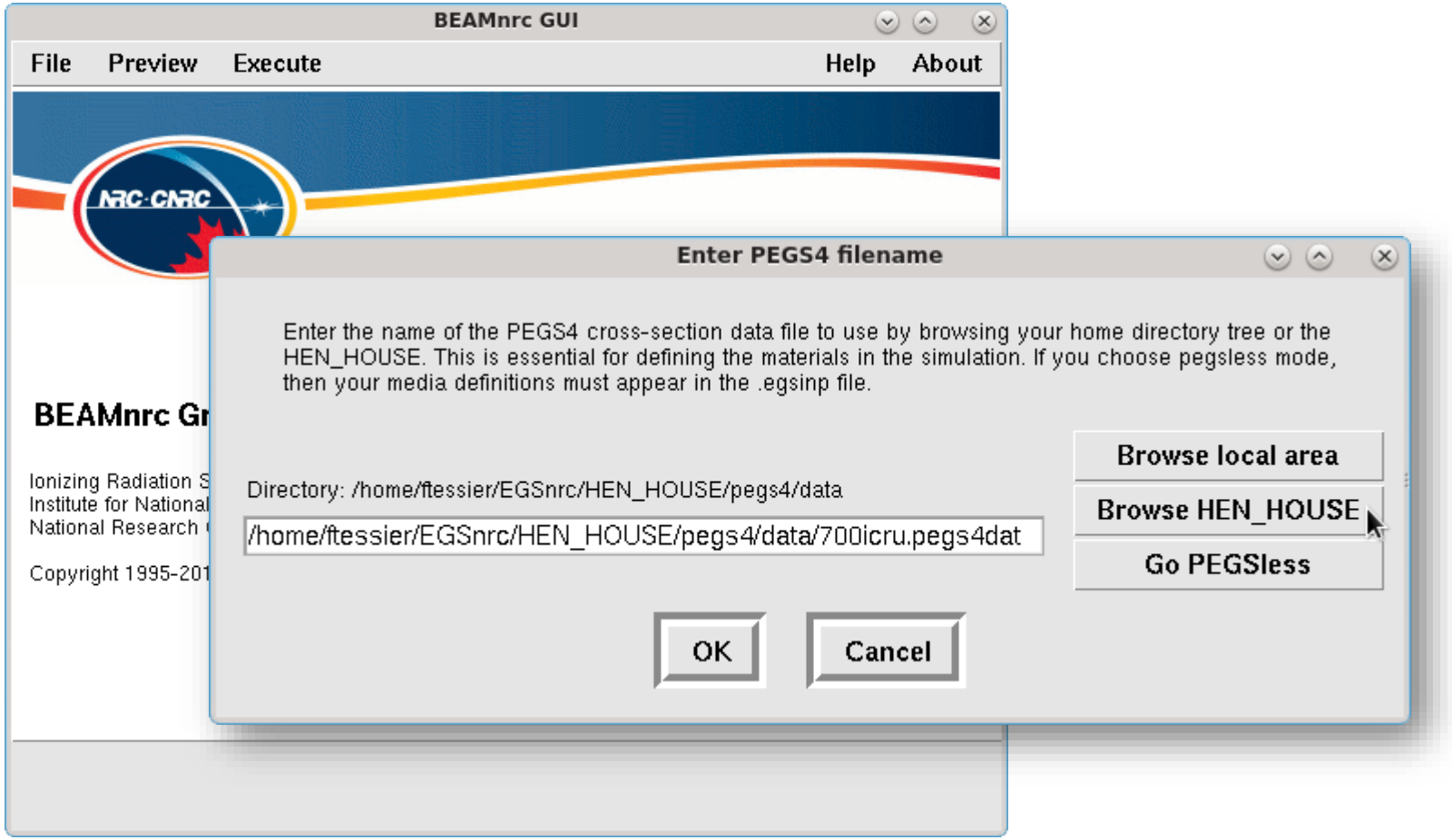
Edit your BEAMnrc input file with `beamnrc_gui`



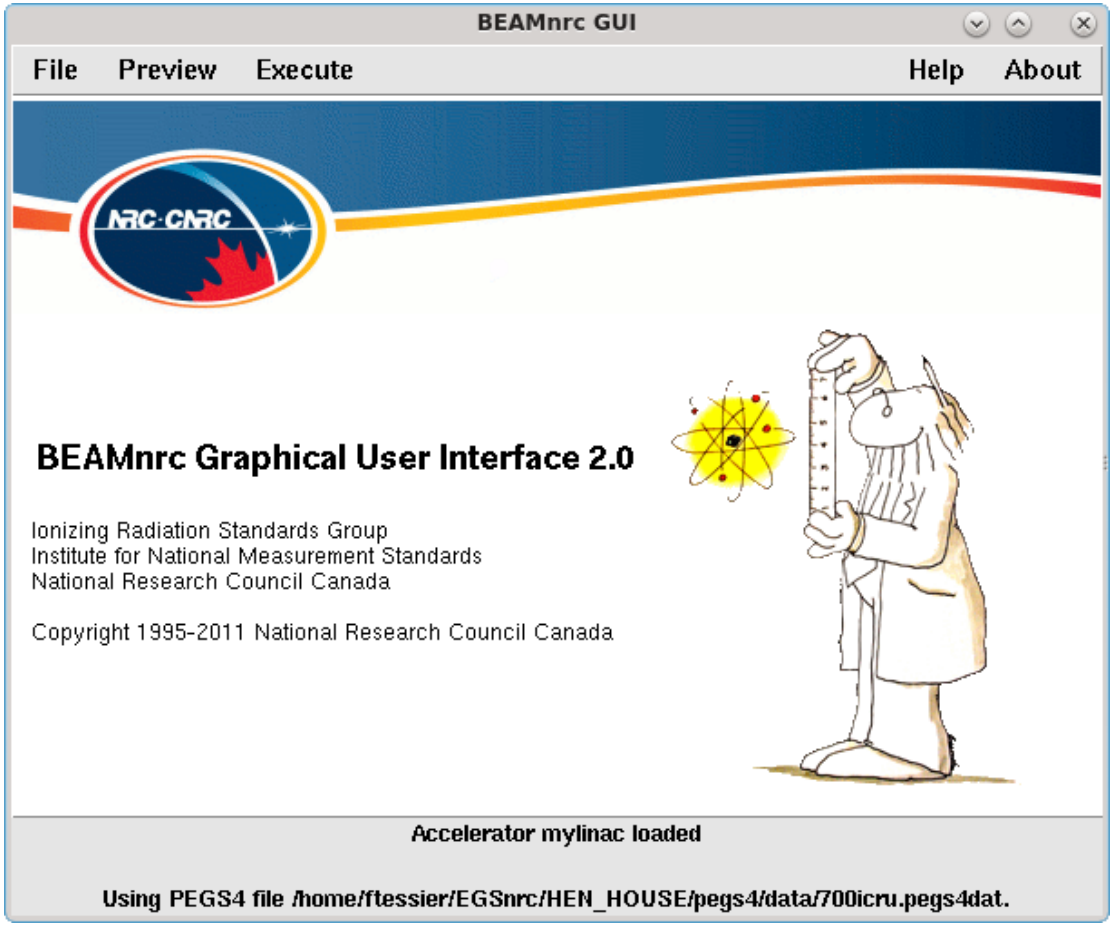
Edit your BEAMnrc input file with `beamnrc_gui`

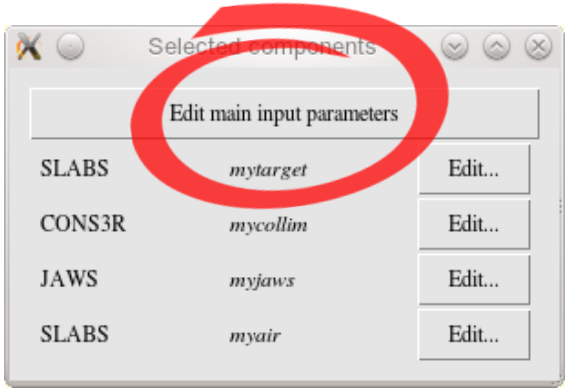


Edit your BEAMnrc input file with `beamnrc_gui`



Edit your BEAMnrc input file with `beamnrc_gui`





BEAMnrc scoring concepts

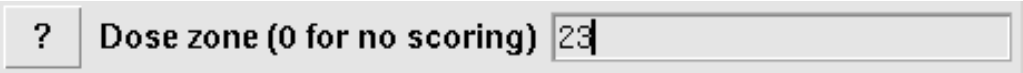
1. **Dose zone:**
   A **collection** of geometrical regions for which dose is reported as a whole at the end of the simulation. You define which geometrical regions are associated to each doze zone in the CM inputs.

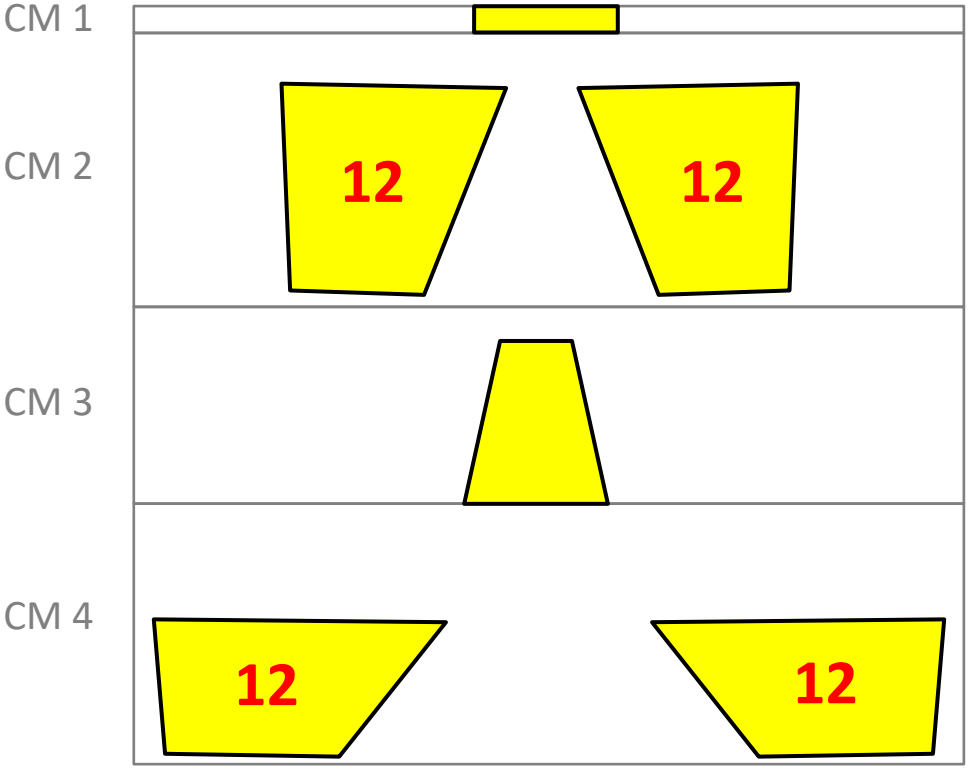
2. **Scoring plane:**
   A plane positioned below any CM to: **a)** calculate number, fluence, average energy and angle, in user-defined planar scoring zone; and **b)** save all particles reaching the plane to a *phase space* file.

3. **Latch bits:**
   The latch variable is a **bit field** travelling with a particle to keep track where the particle and its ancestors have been, or where they have interacted.

1. Dose zone

Somewhere in the CM input, for each geometrical region, there is a **dose zone input:**



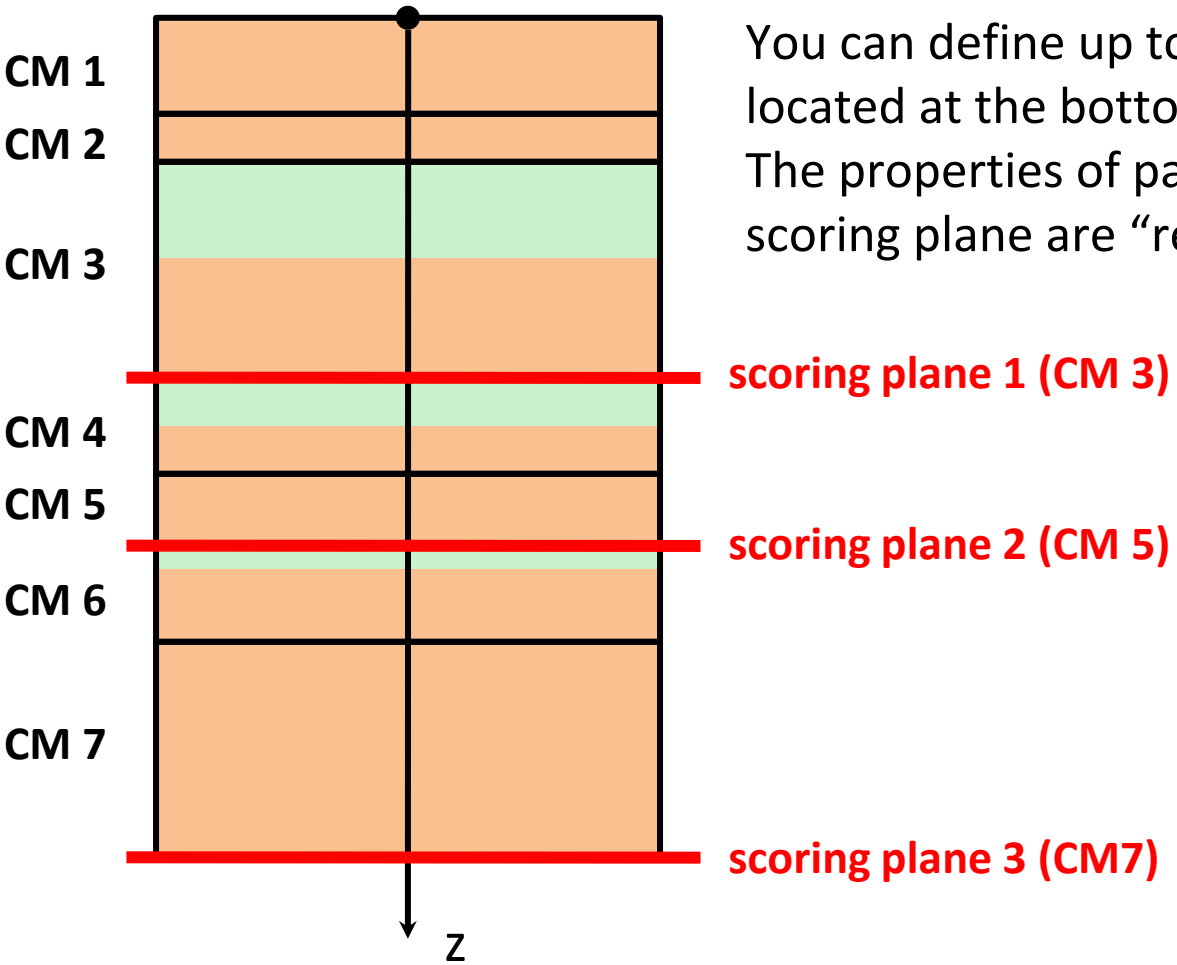


Suppose I set the dose zone to **12** for all 4 jaw regions in this diagram.

Then BEAMnrc will output a dose summary for the dose zone 12, which will give the **dose deposited in all 4 regions, as a whole.**

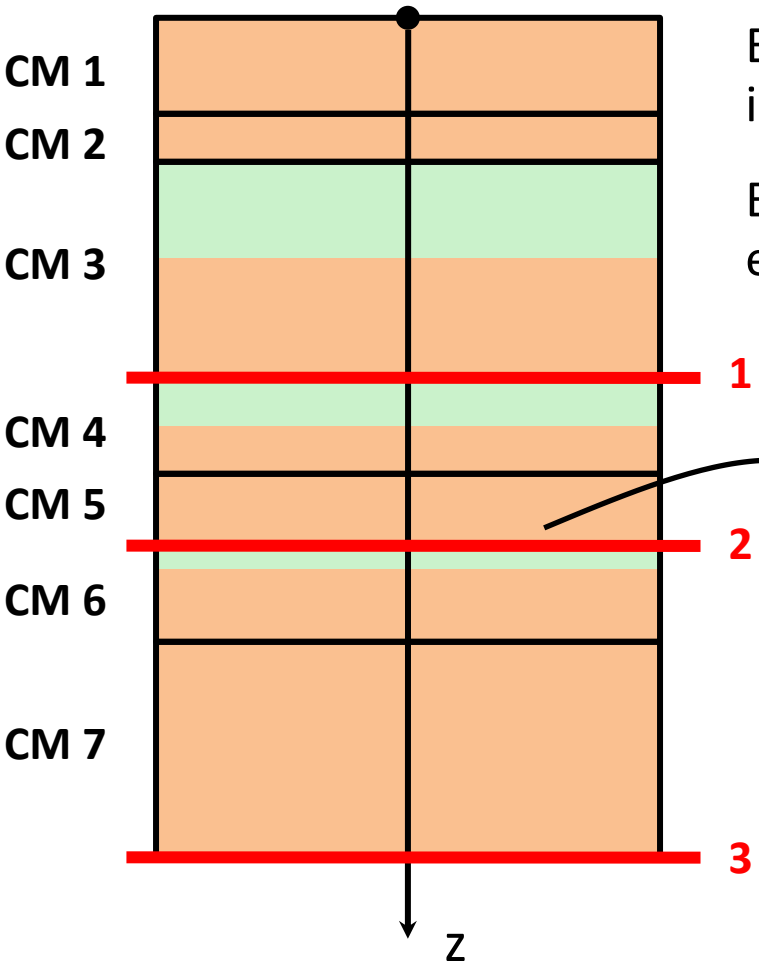
(Dose zones don't have to be assigned.)

2. Scoring plane: a) analyze fluence



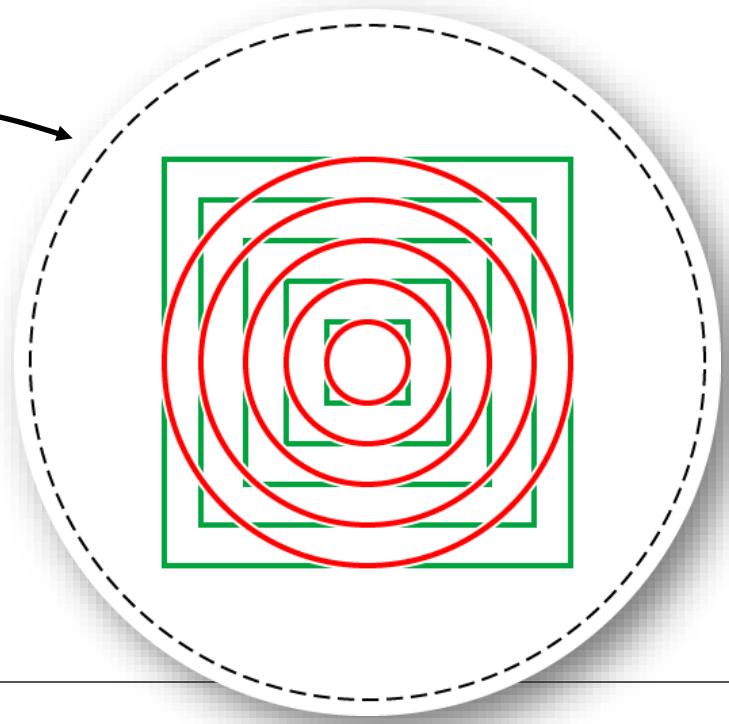
You can define up to **3 scoring planes,** located at the bottom of any CM.
The properties of particles reaching a scoring plane are "recorded".

2. Scoring plane: a) analyze fluence

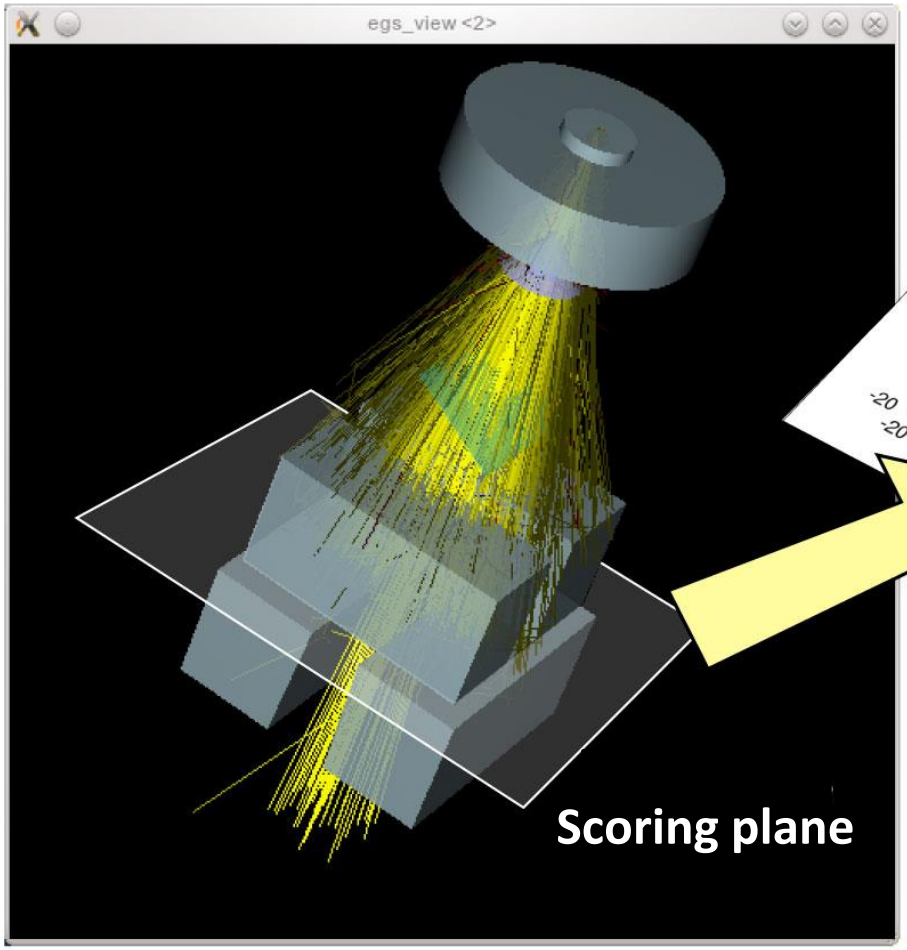


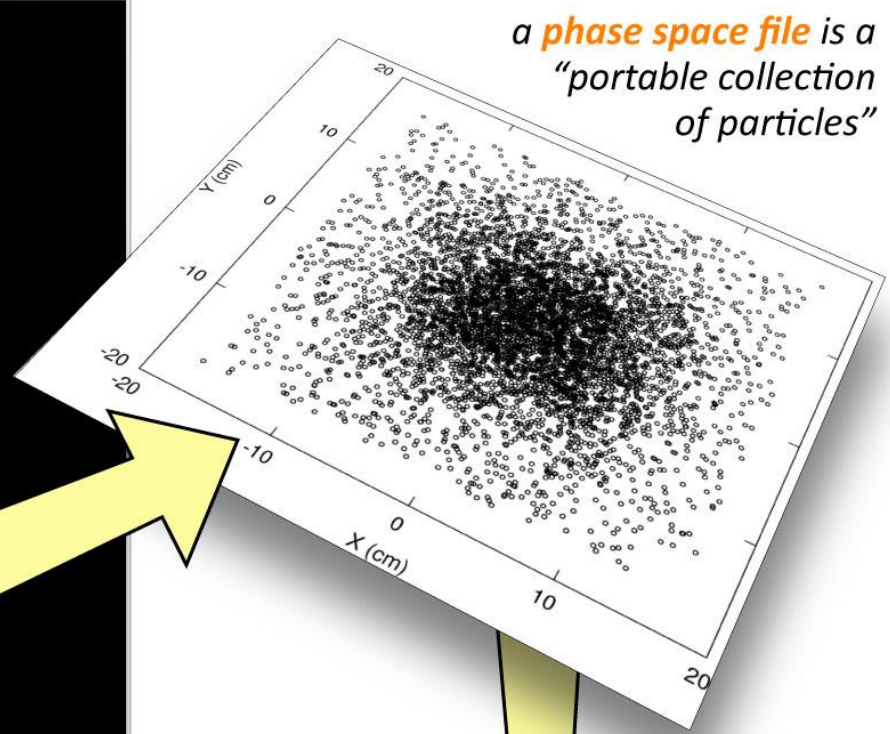
Each scoring plane can be further divided in radial or square **scoring zones.**

BEAMnrc reports number, fluence, average energy and angle for each scoring zone.



2. Scoring plane: b) save particles to file





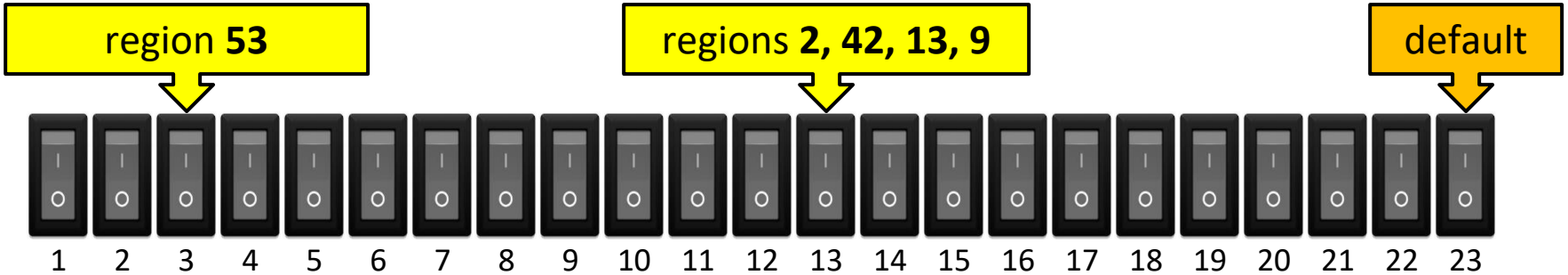
*a **phase space file** is a "portable collection of particles"*

input for another simulation

3. “Latch” but not least: latch bits

If you can operate a light switch, you understand latch! The latch variable is a set of 32 bits that particles carry around with them. We care about **bits 1–23.**

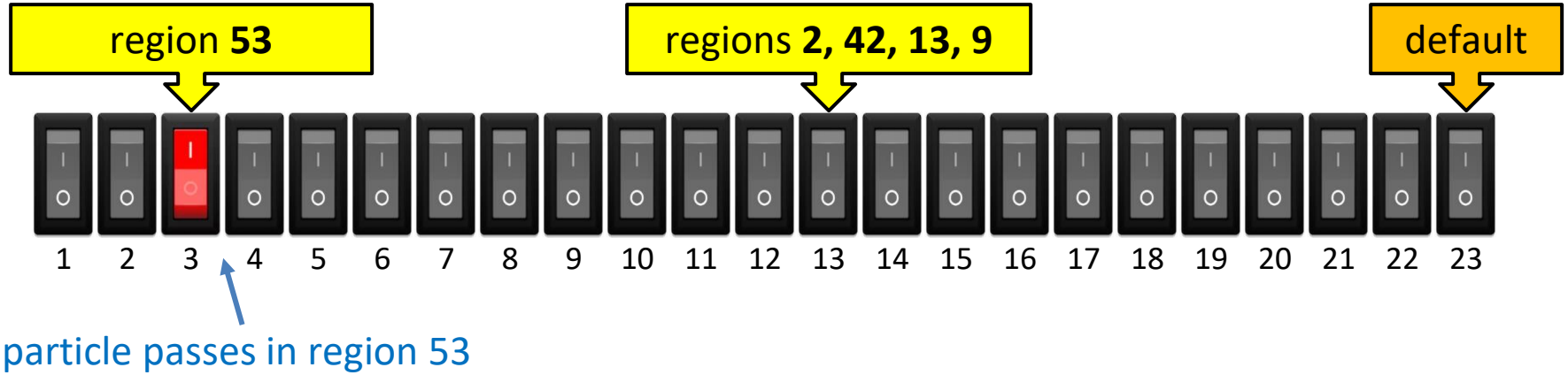
When you design a component module, you can associate any geometrical region with any given latch bit:



3. “Latch” but not least: latch bits

If you can operate a light switch, you understand latch! The latch variable is a set of 32 bits that particles carry around with them. We care about **bits 1–23.**

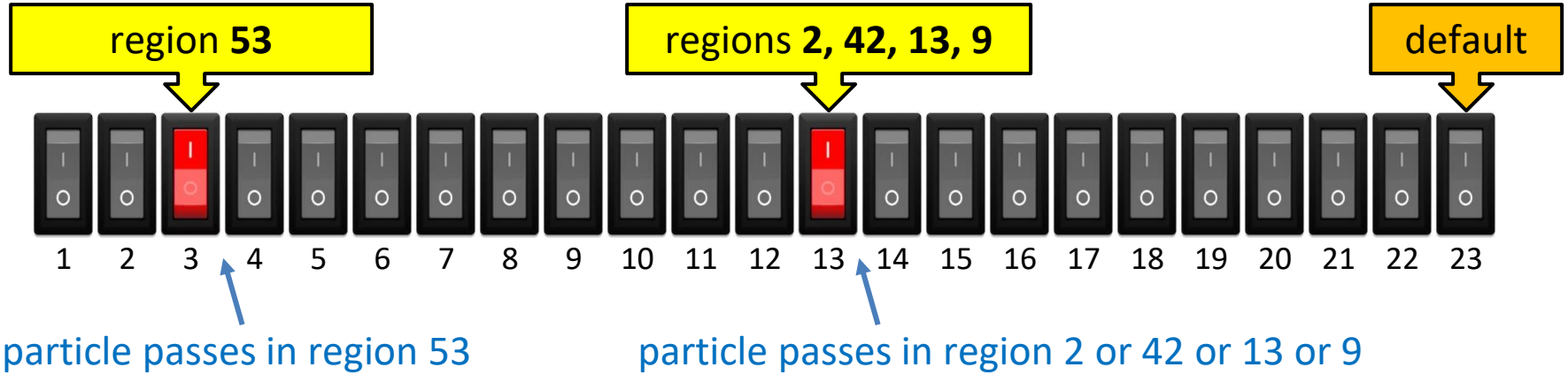
When you design a component module, you can associate any geometrical region with any given latch bit:



3. “Latch” but not least: latch bits

If you can operate a light switch, you understand latch! The latch variable is a set of 32 bits that particles carry around with them. We care about **bits 1–23.**

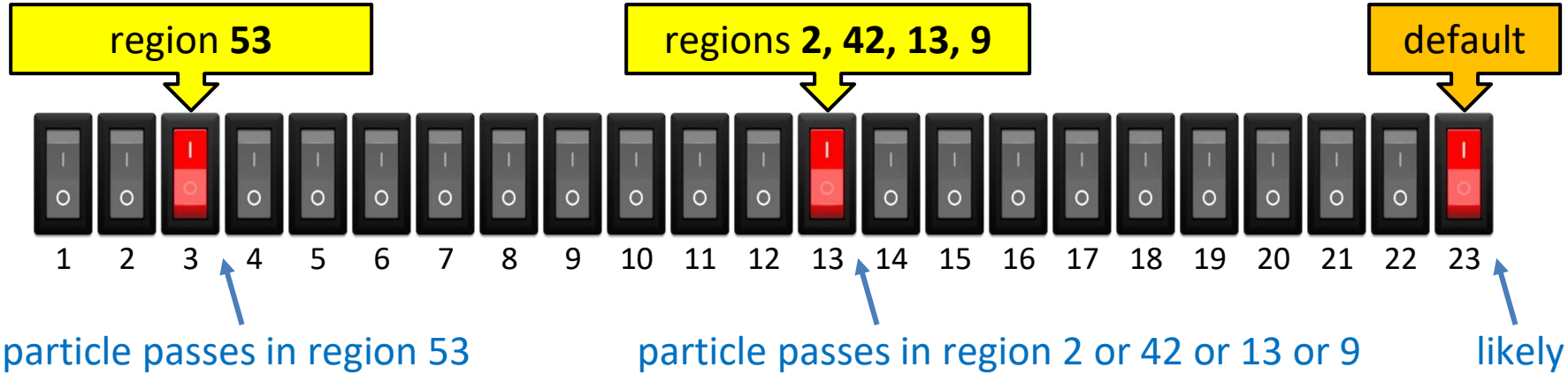
When you design a component module, you can associate any geometrical region with any given latch bit:



3. “Latch” but not least: latch bits

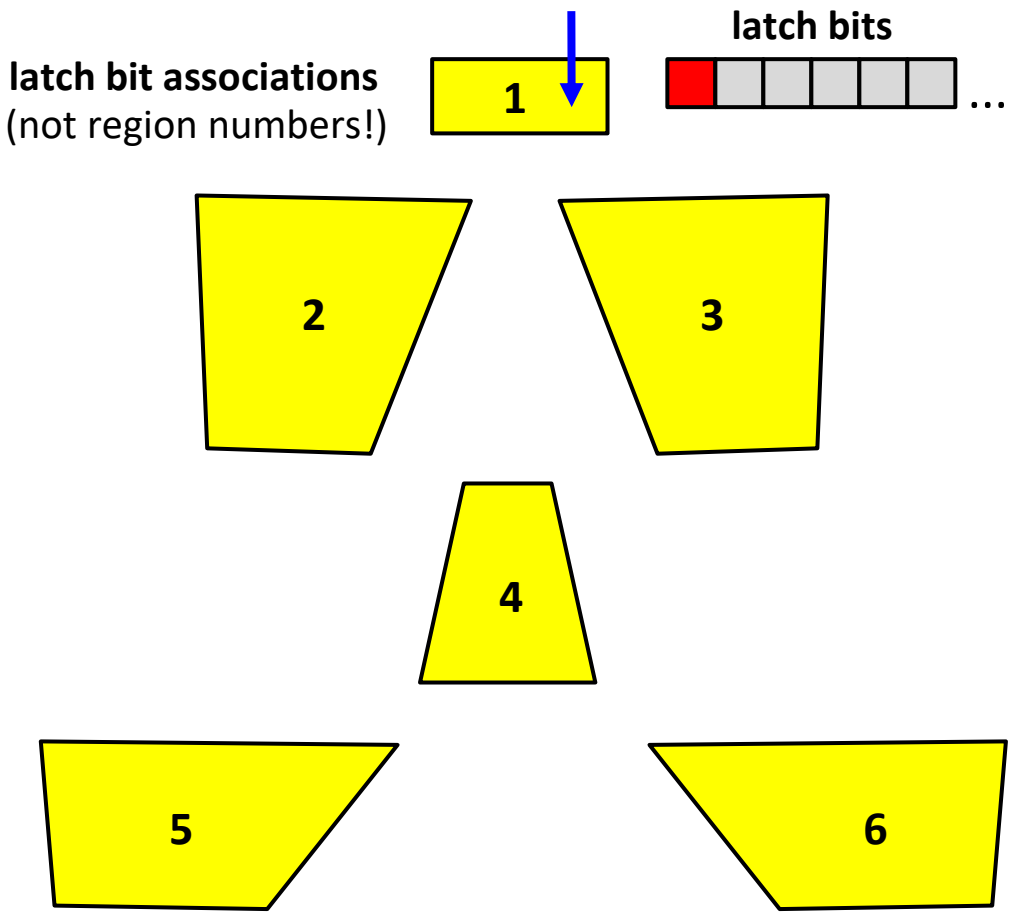
If you can operate a light switch, you understand latch! The latch variable is a set of 32 bits that particles carry around with them. We care about **bits 1–23.**

When you design a component module, you can associate any geometrical region with any given latch bit:



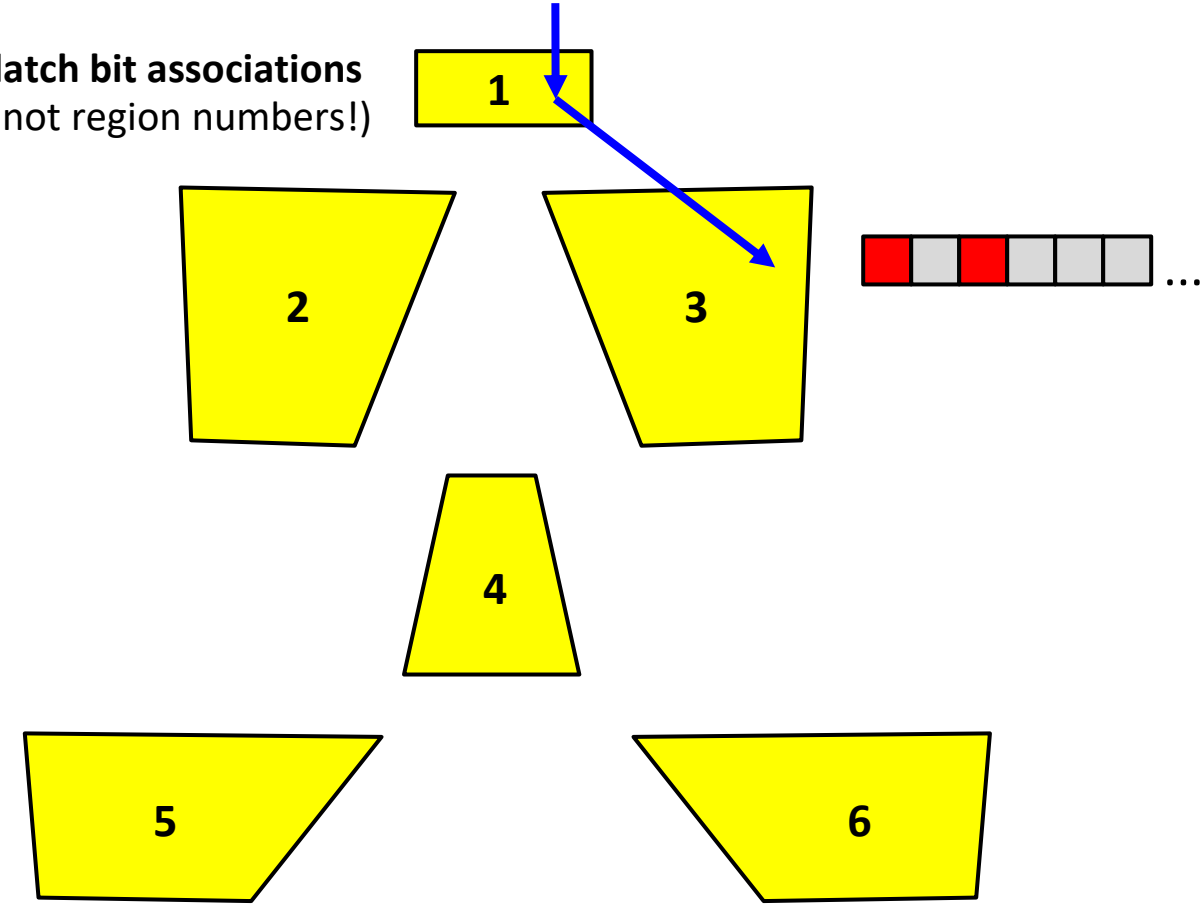
If the particle passes through any region associated with a latch bit, this bit is turned on (set to 1). **This is a bit filter for dose components and analysis!**

3. “Latch” but not least: latch inheritance

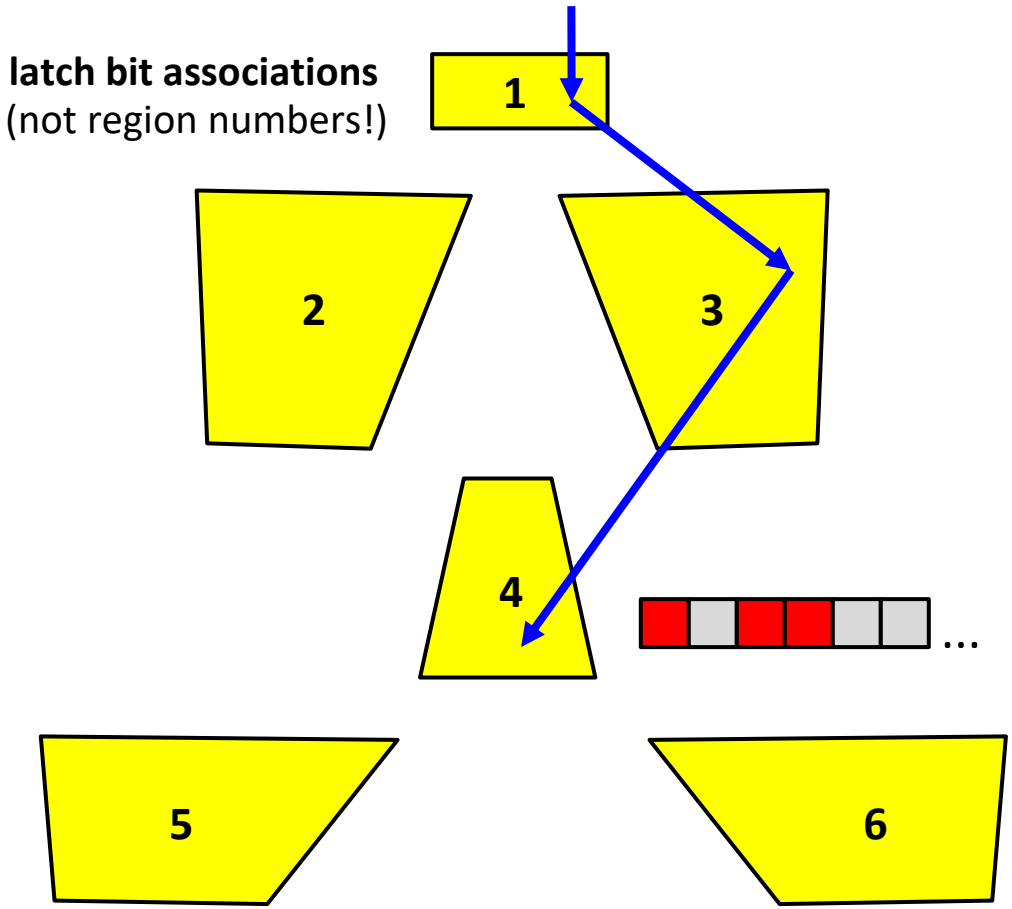


3. “Latch” but not least: latch inheritance

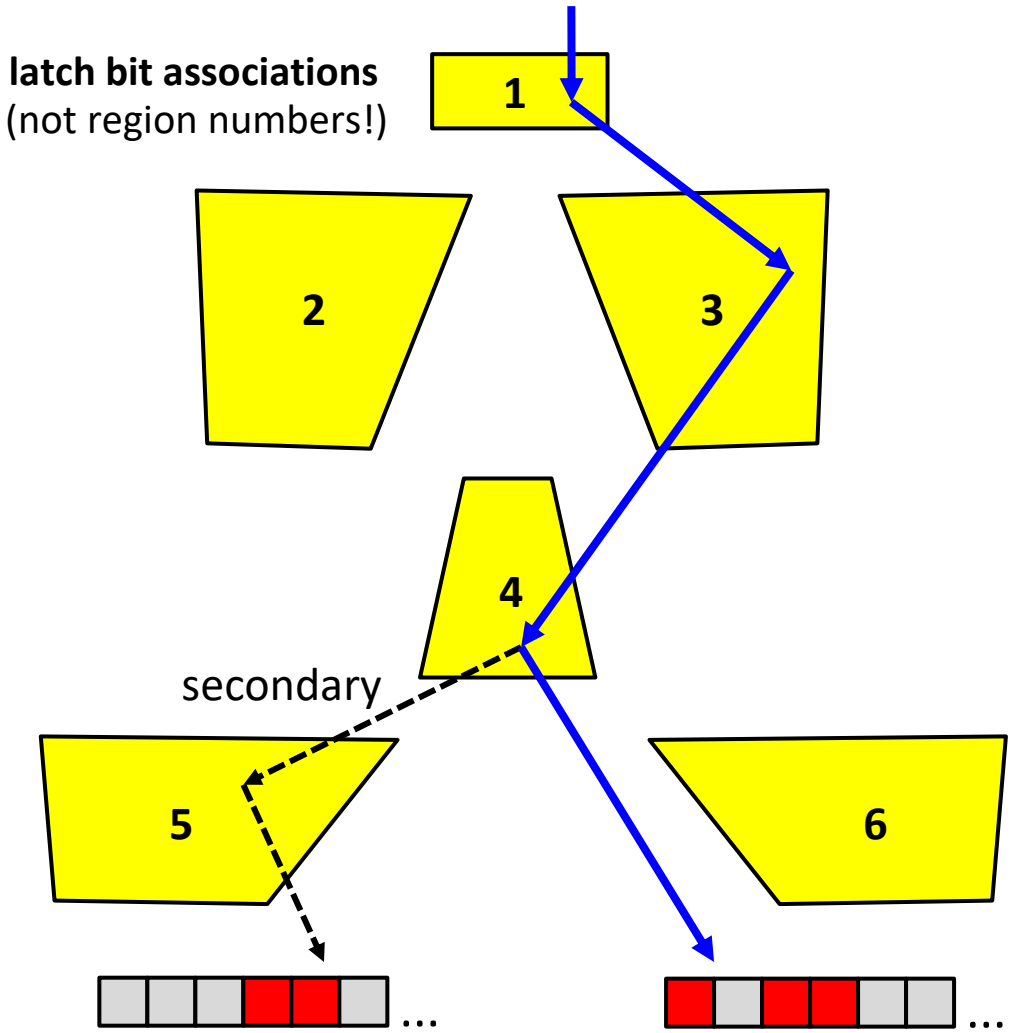
latch bit associations
(not region numbers!)



3. “Latch” but not least: latch inheritance

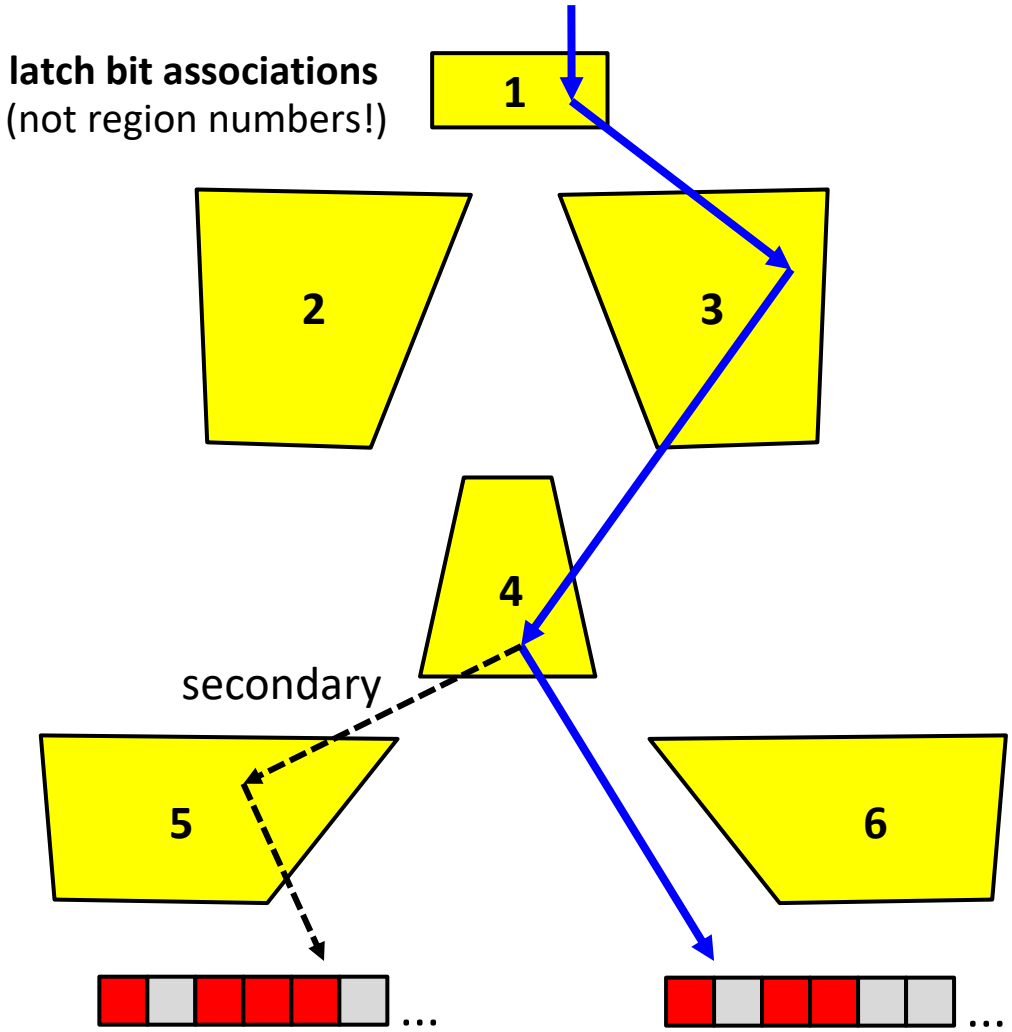


3. “Latch” but not least: latch inheritance



non-inherited latch:
bits are set by passage, and no latch information is passed on to secondaries.

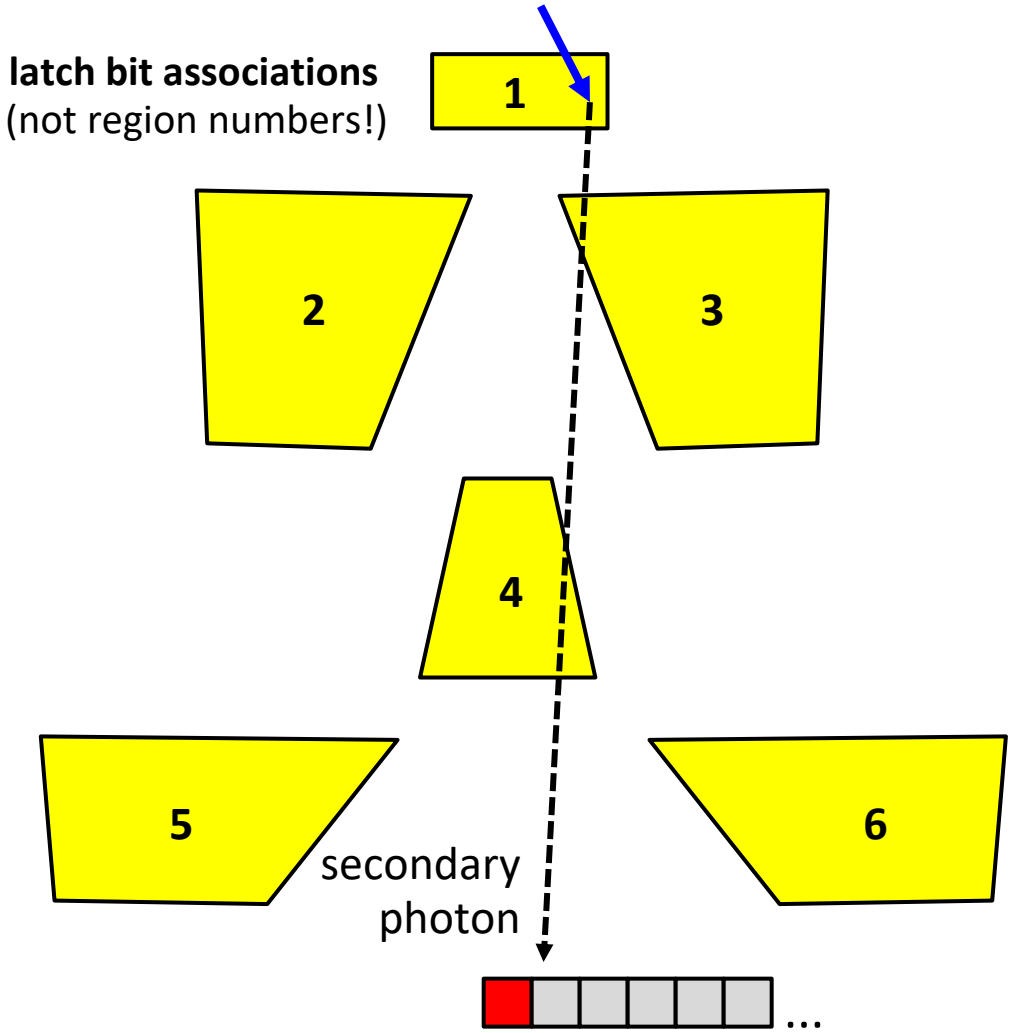
3. “Latch” but not least: latch inheritance



non-inherited latch:
bits are set by passage, and no latch information is passed on to secondaries.

inherited latch, by passage:
bits are set by passage, and latch is **copied** to newly created secondary particles.

3. “Latch” but not least: latch inheritance

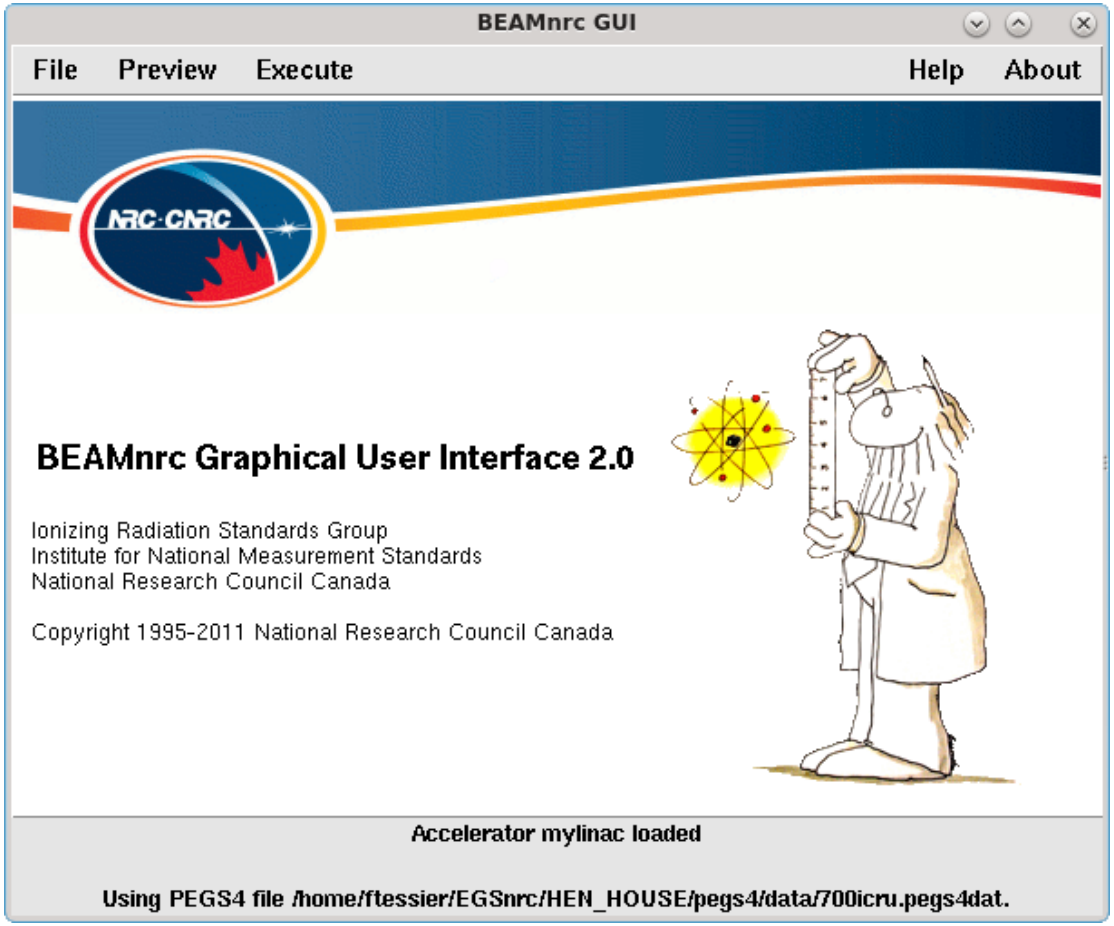


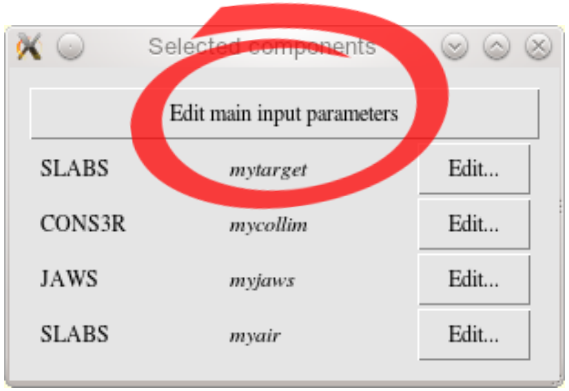
non-inherited latch:
bits are set by passage, no latch information is passed on to secondaries.

inherited latch, by passage:
bits are set by passage, and latch is **copied** to newly created secondary particles.

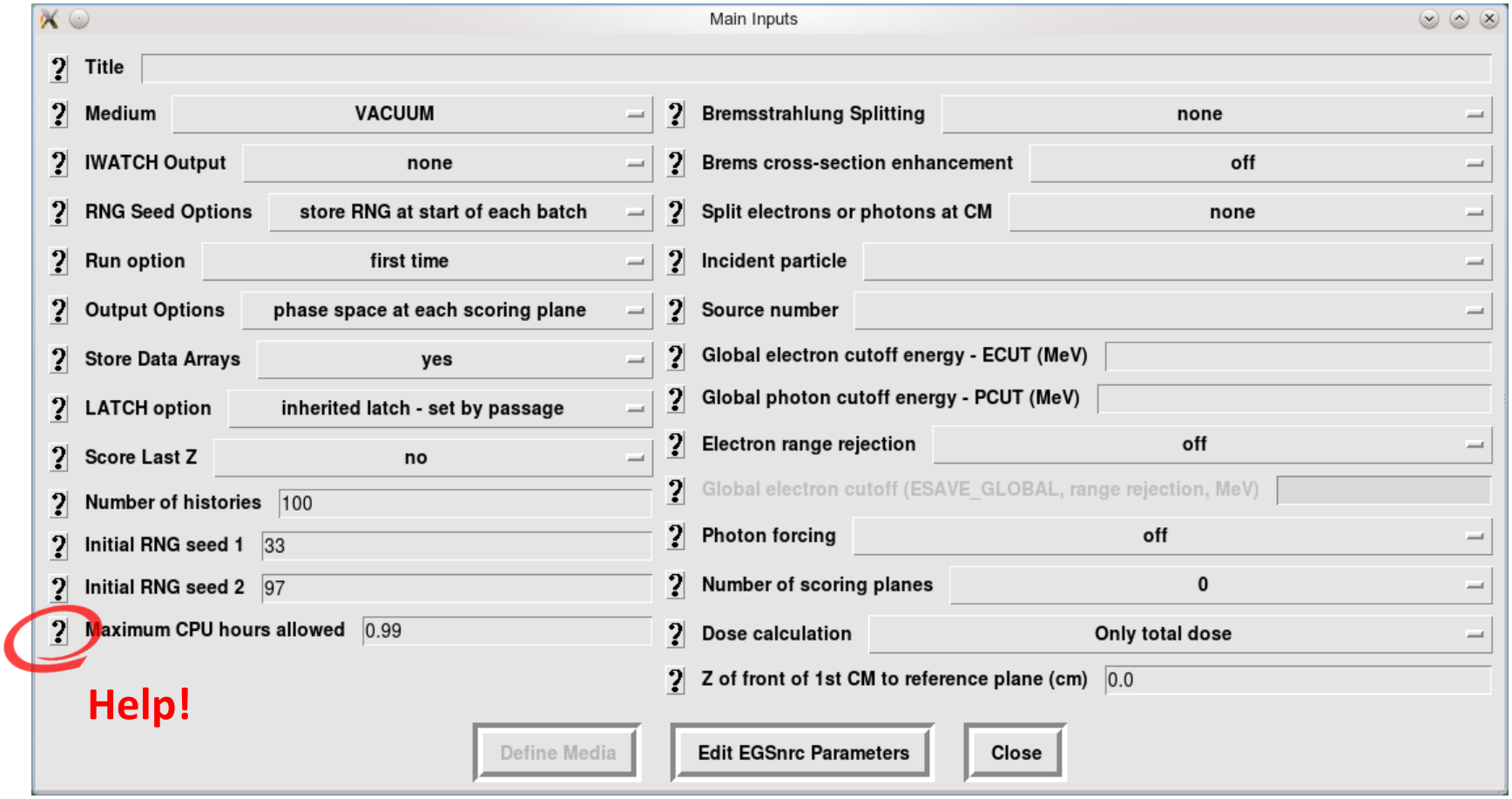
inherited latch, by interaction:
for <u>photons</u>, a bit is only set if the photon **interacts** in the associated region(s).

Edit your BEAMnrc input file with `beamnrc_gui`

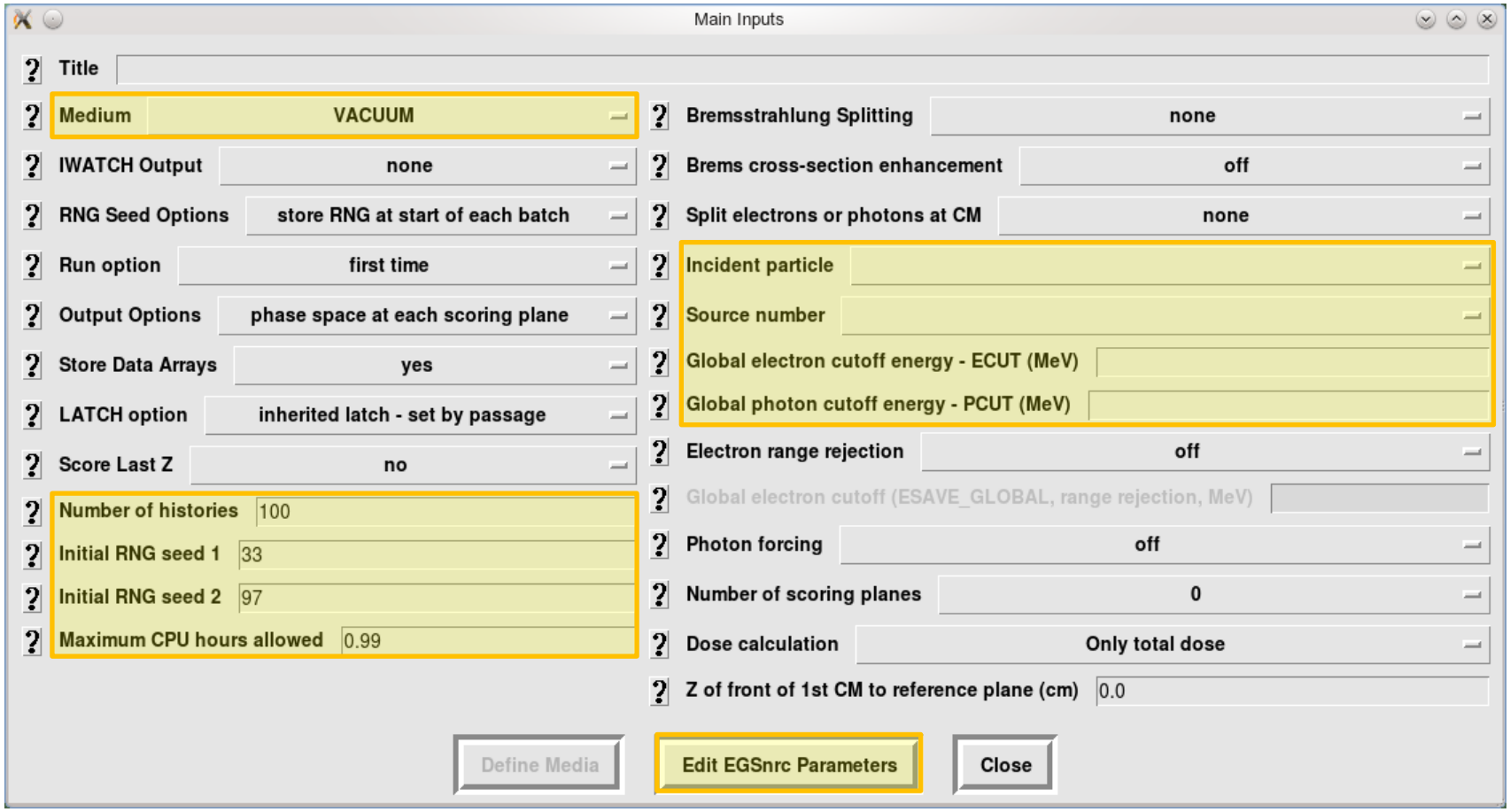




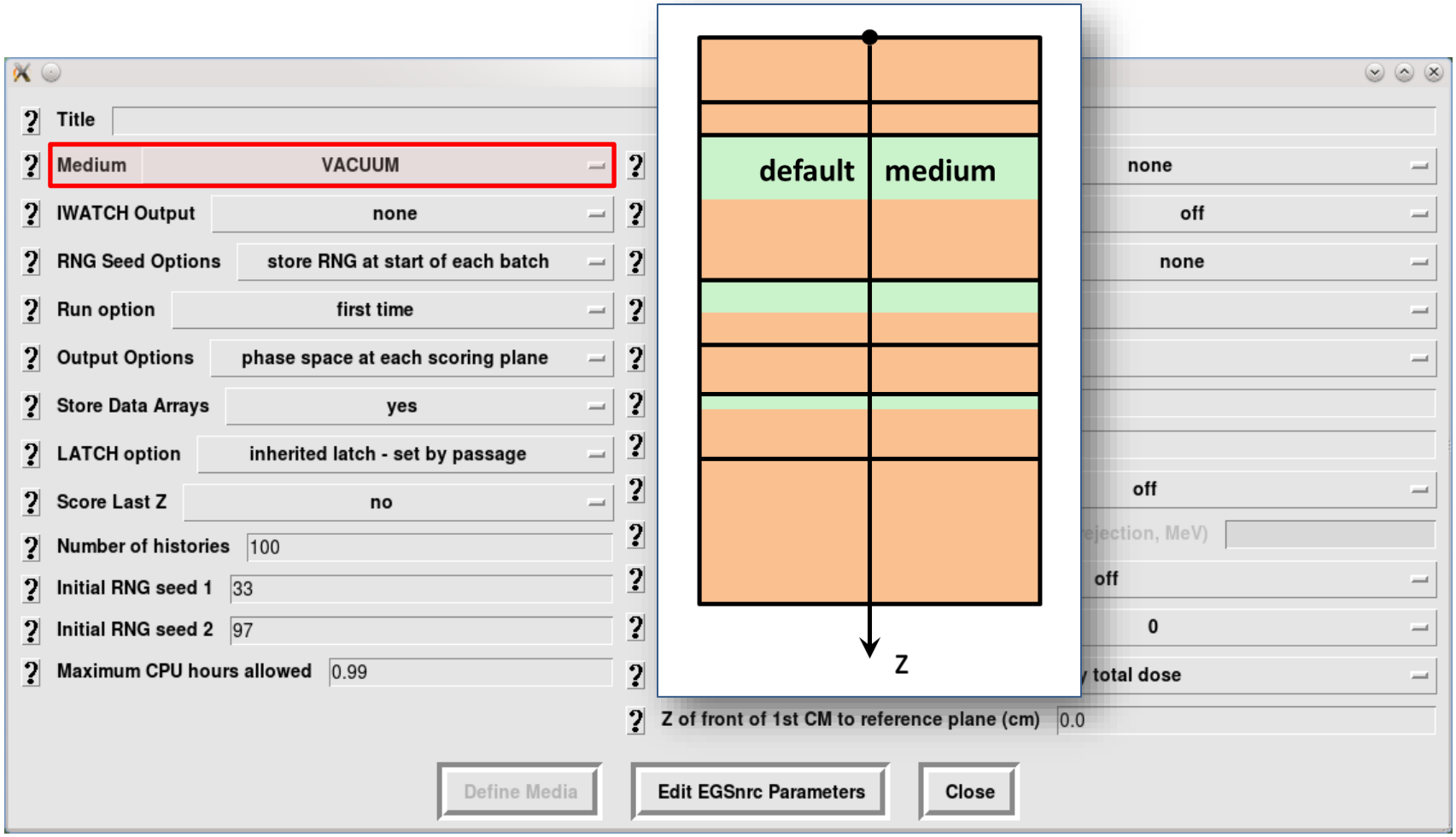
BEAMnrc main inputs: *everything* except geometry



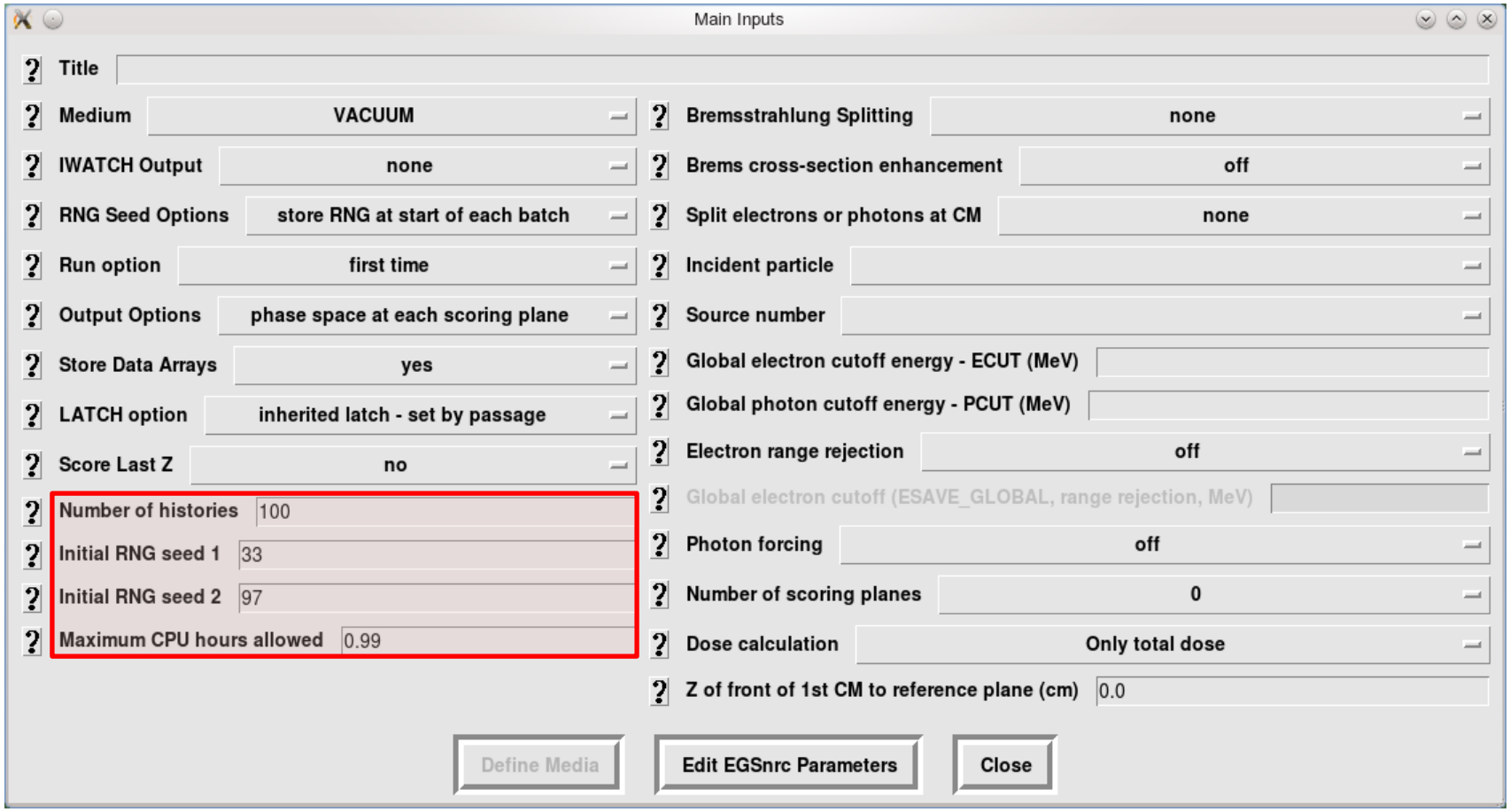
BEAMnrc main inputs: the “easy” ones



Default medium

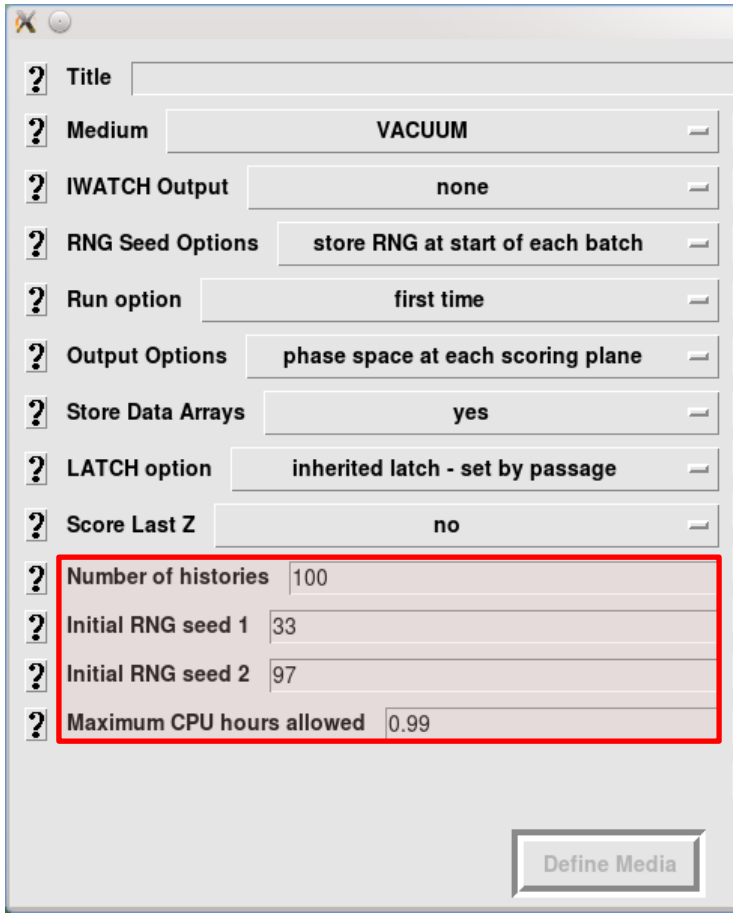


Number of histories and seeds



NCASE

Number of histories and seeds



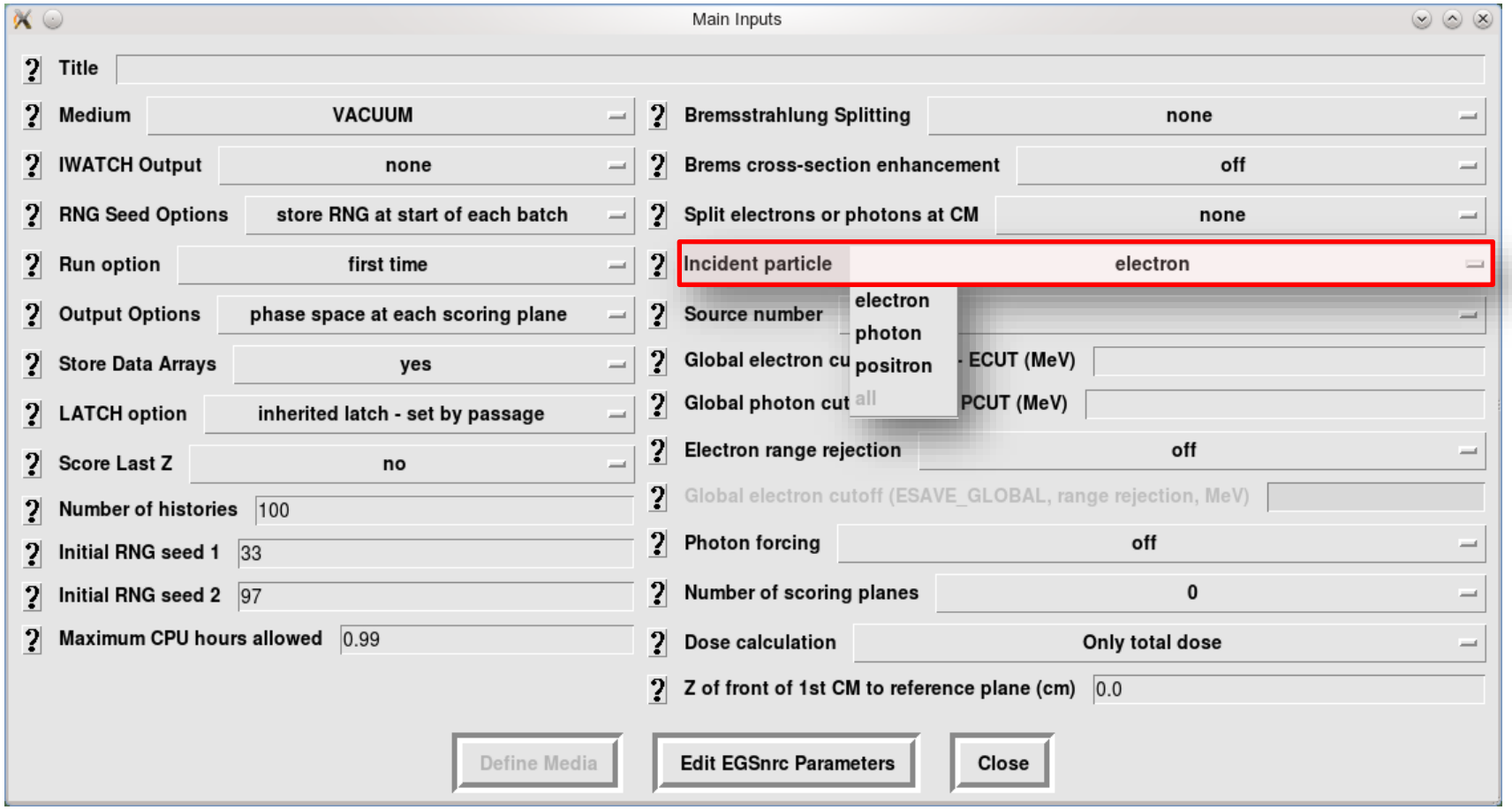
NCASE

$$t \sim N$$

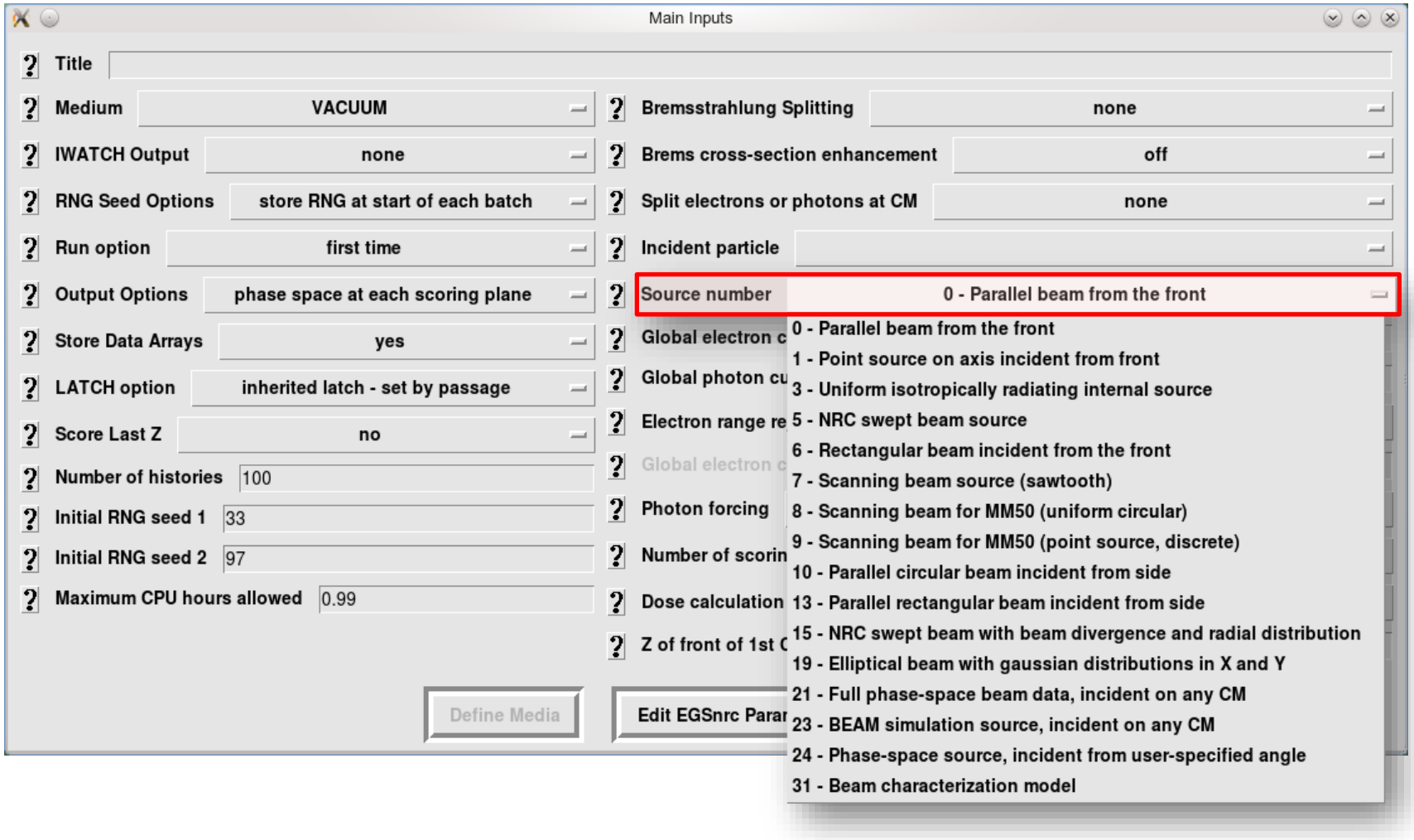
$$\sigma \sim 1/\sqrt{N}$$

Remember:
using the same seeds will run exactly the same simulation (on the same hardware)

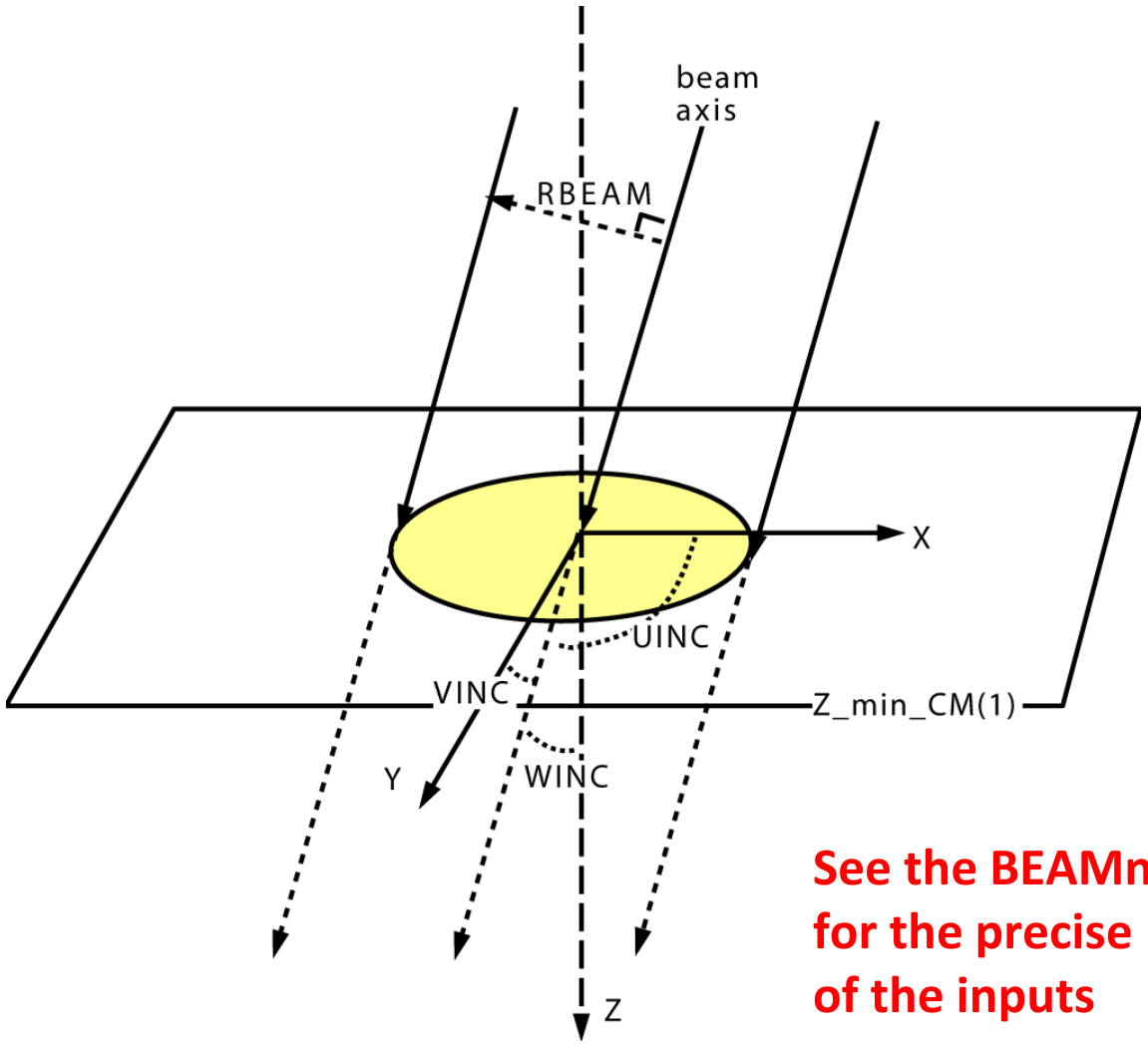
Source particle can be electron, photon or positron



BEAMnrc offers 16 different source types

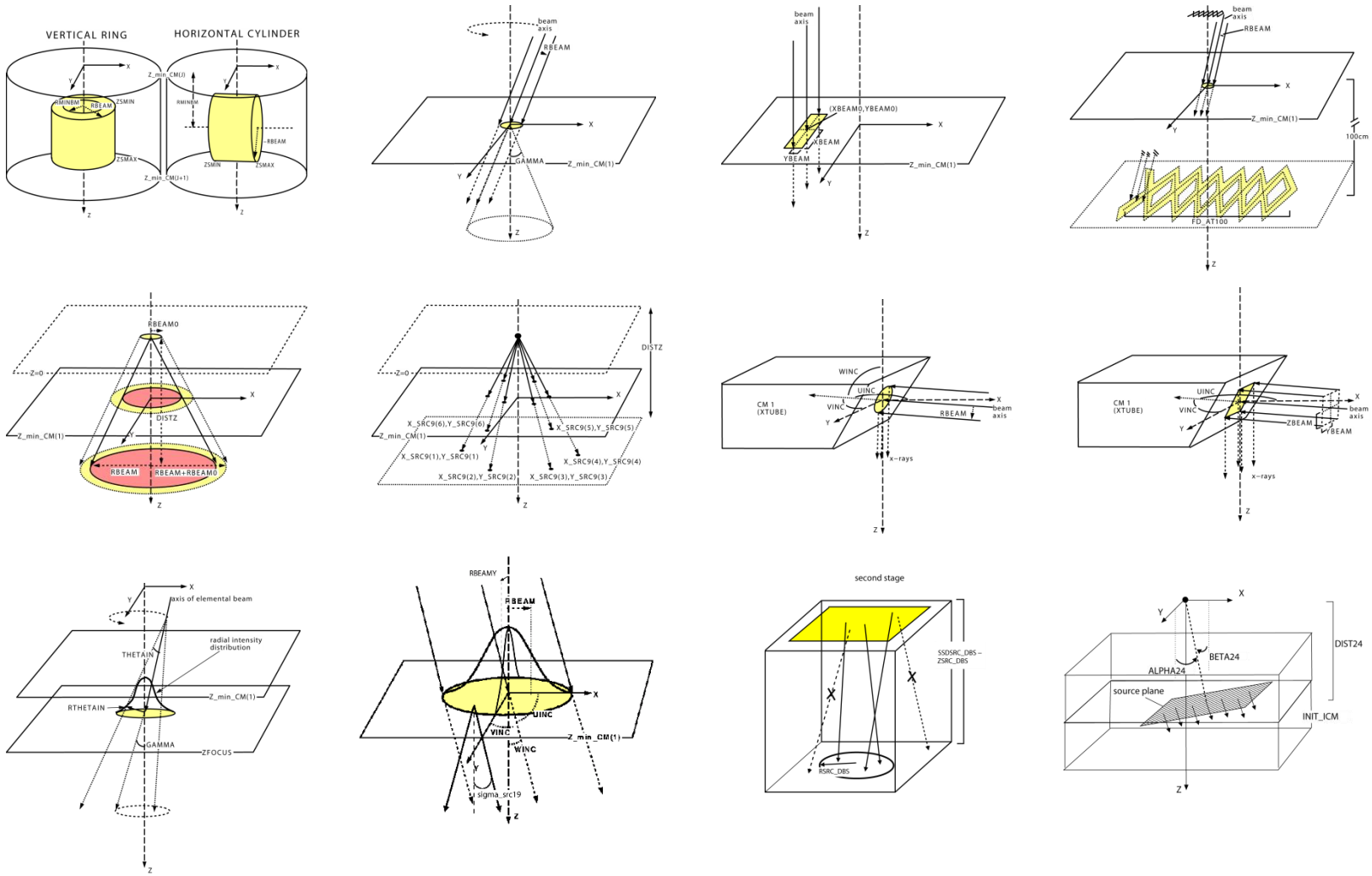


Example: source 0, parallel beam from the front

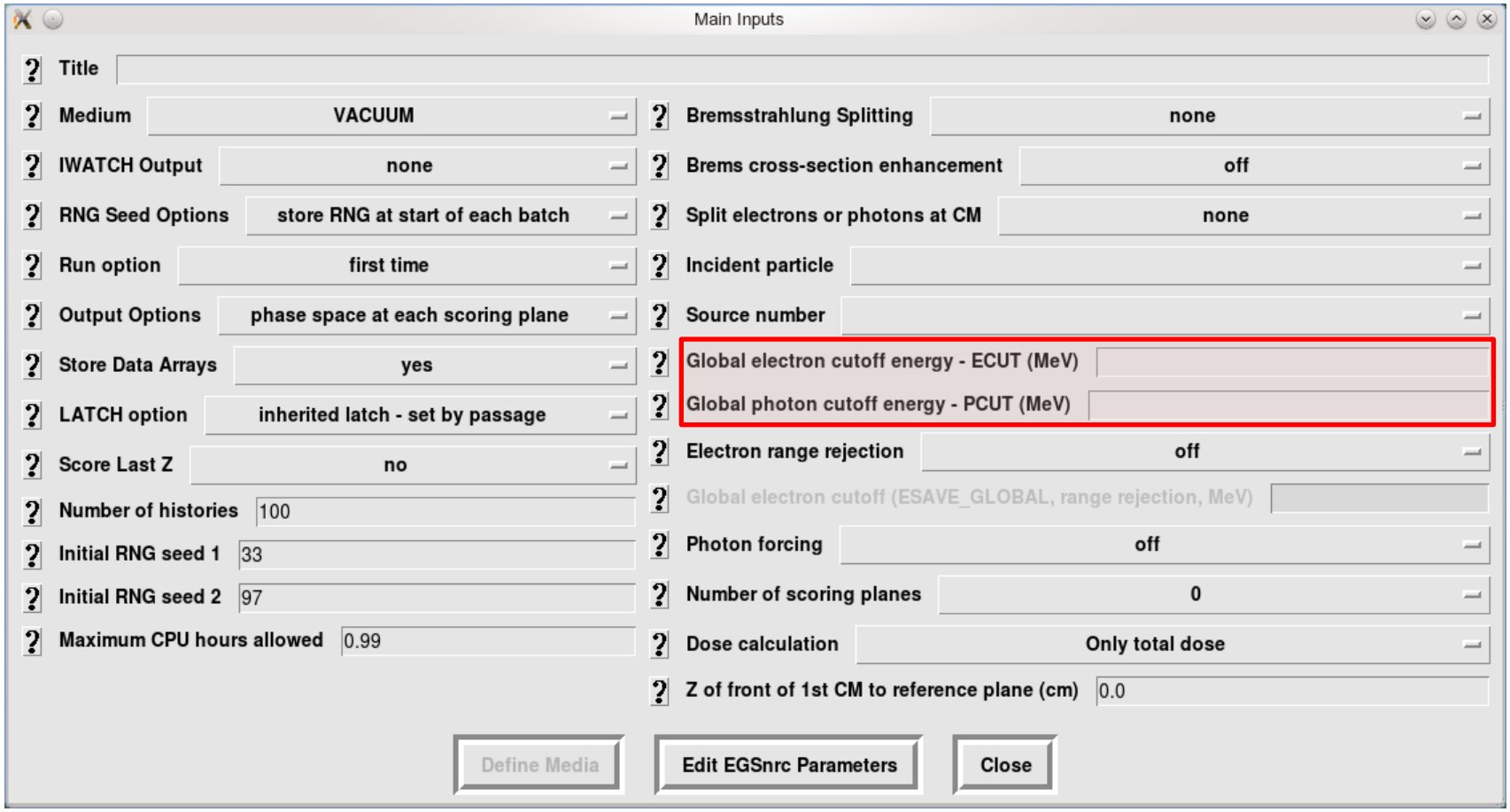


See the BEAMnrc manual for the precise meaning of the inputs

Sources will be discussed in the next lecture

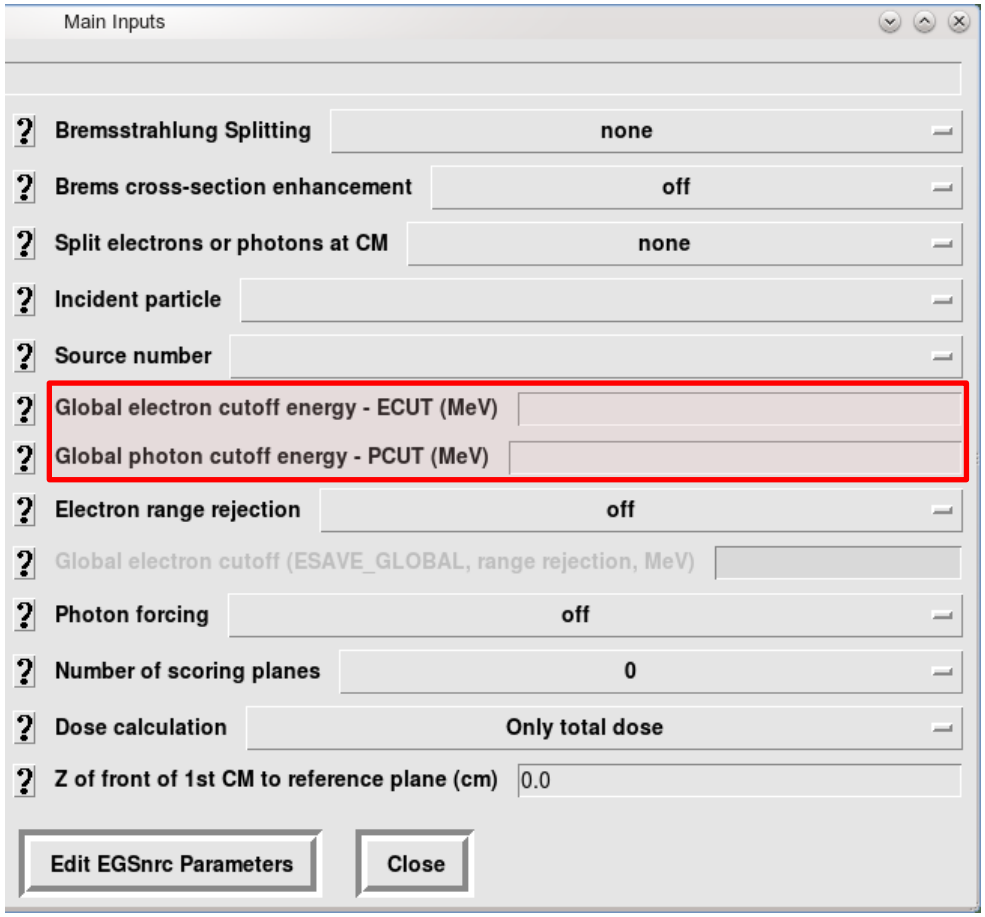


Default ECUT and PCUT

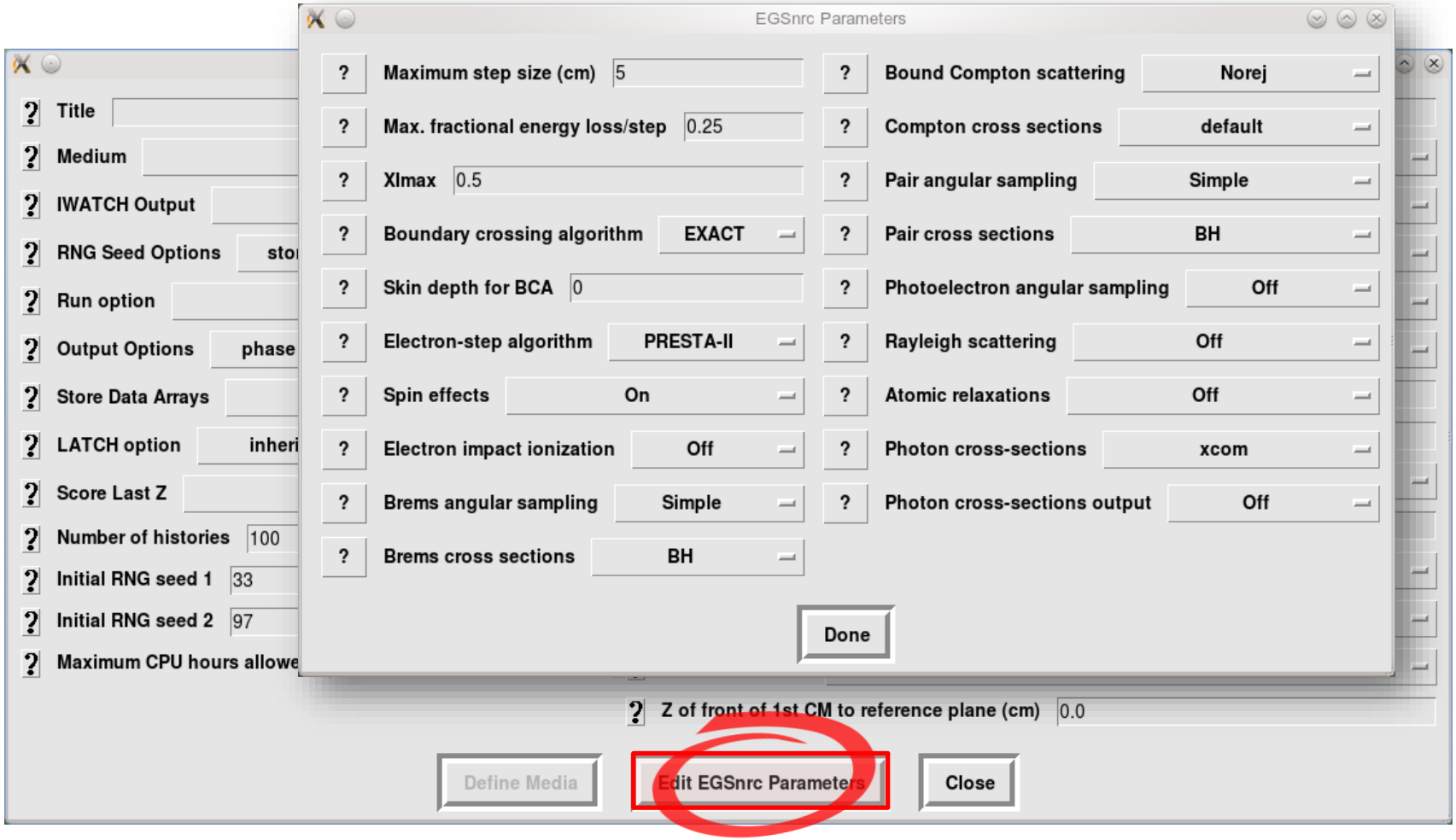


Default ECUT and PCUT

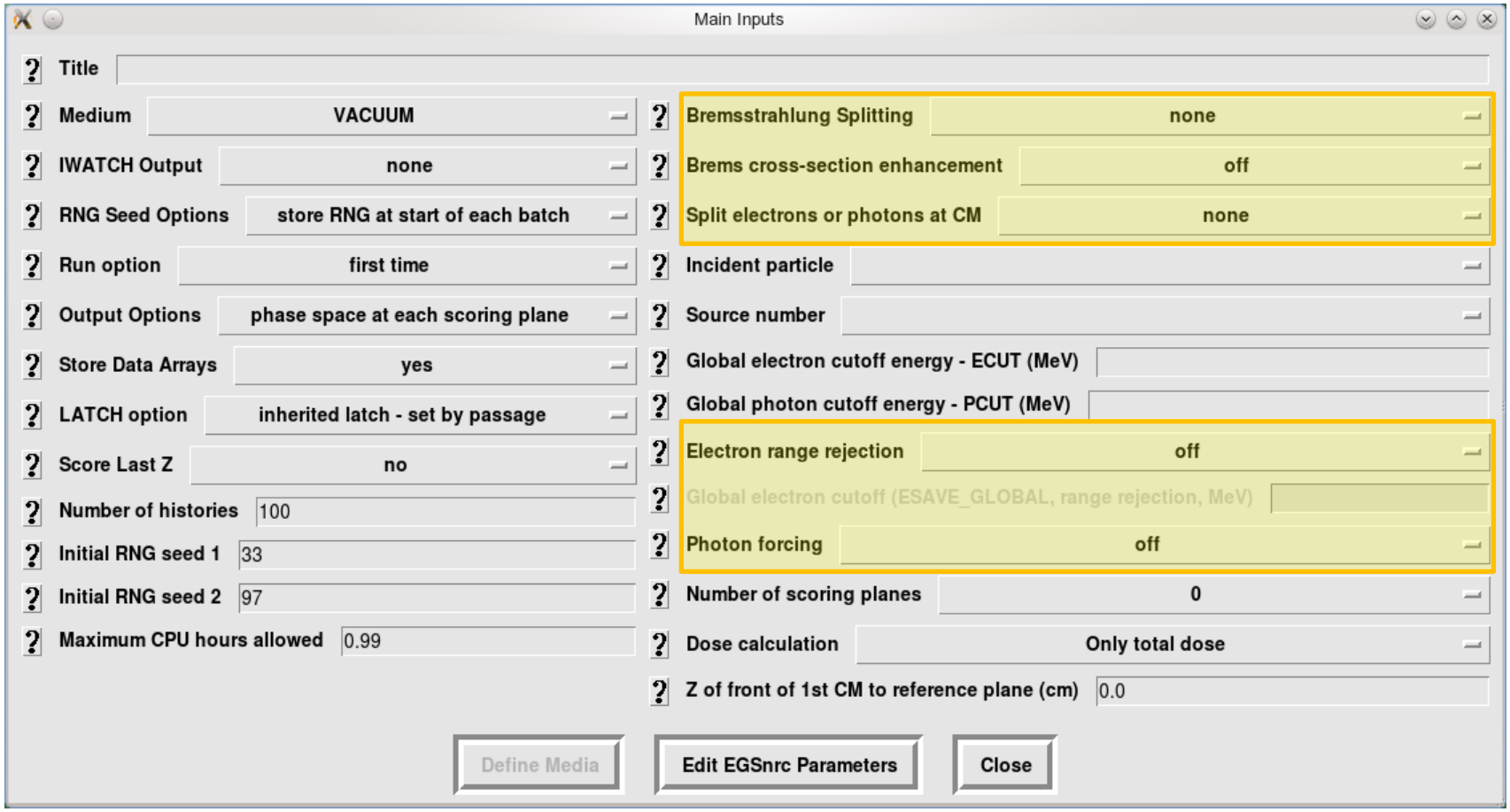
- Global **ECUT** and **PCUT** are applied to all regions by default. Input value is in MeV.
- They can be **overridden** (increased) in each region, via CM input parameters.
- It is common in linacs to have **larger cutoff values** because we care about what reaches the bottom of the linac.
- For example, **ECUT = 0.700 PCUT = 0.189** is a common choice (this is why there is a 700icru pegs data set!)



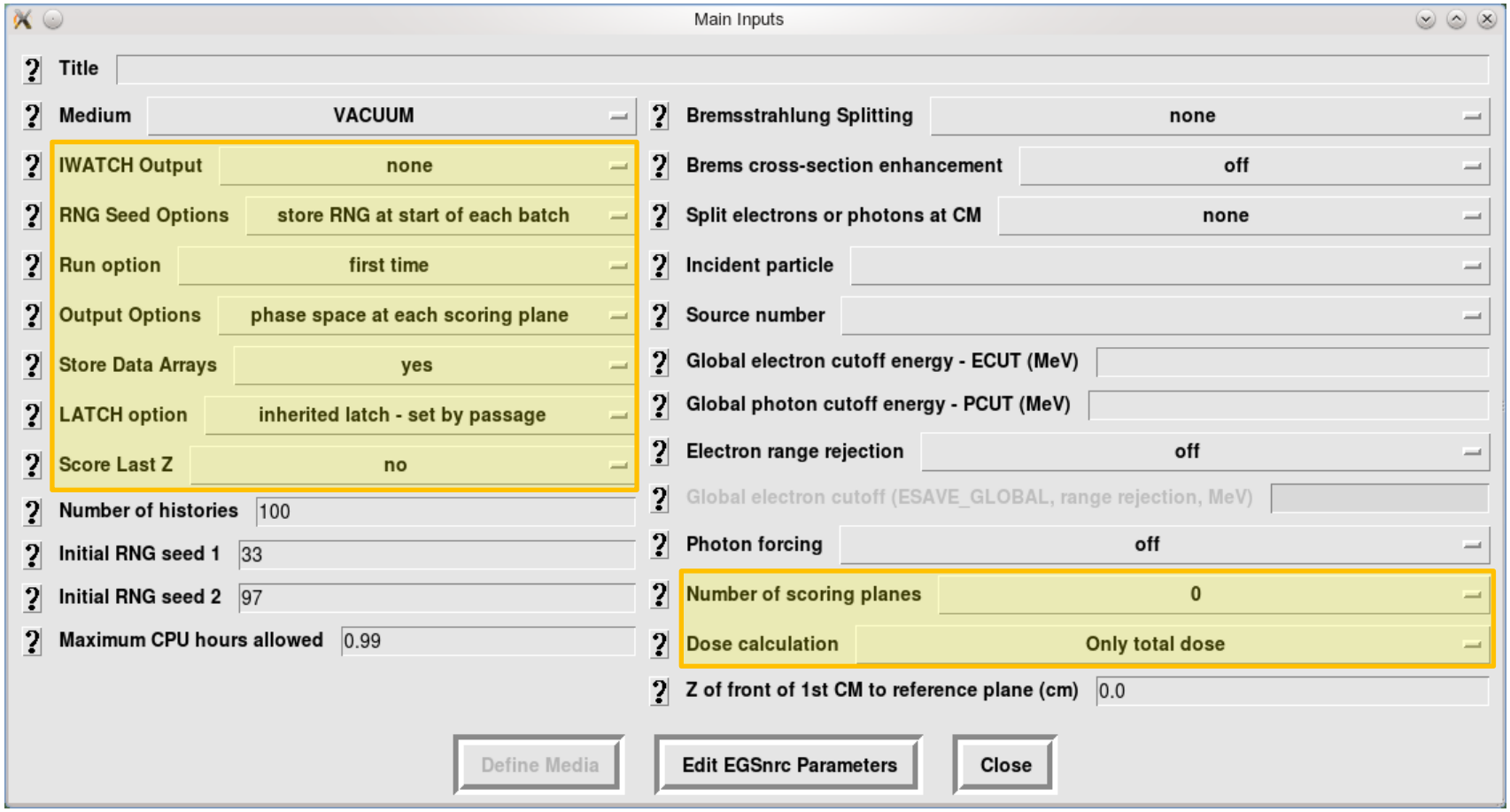
EGSnrc parameters discussed in previous lectures



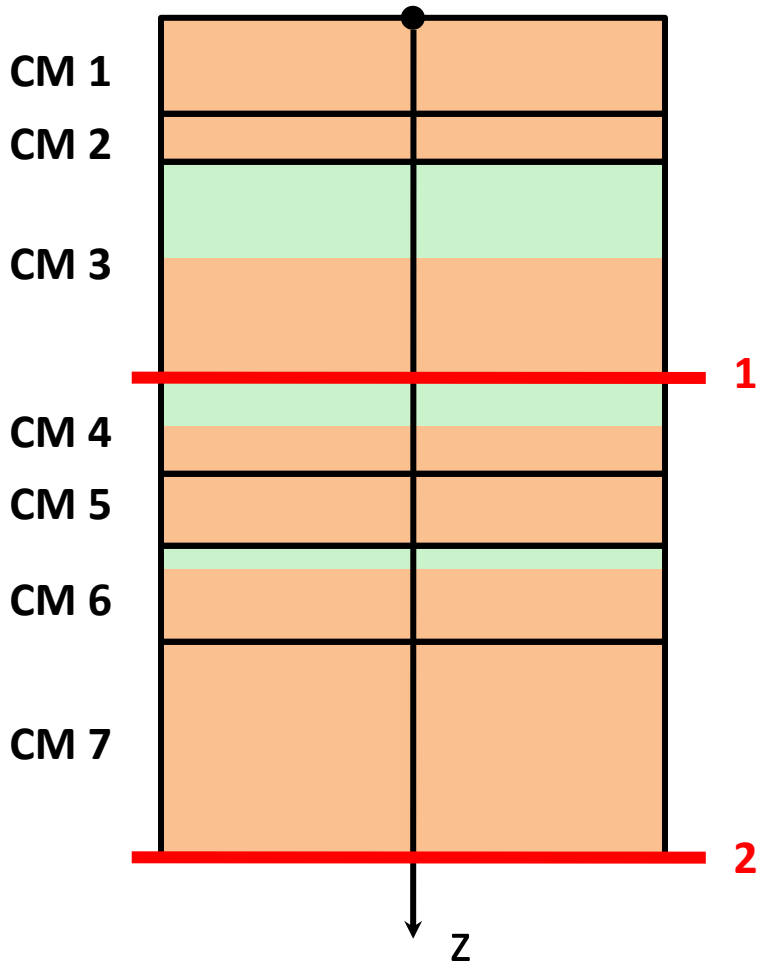
BEAMnrc main inputs: VRTs discussed in other lecture

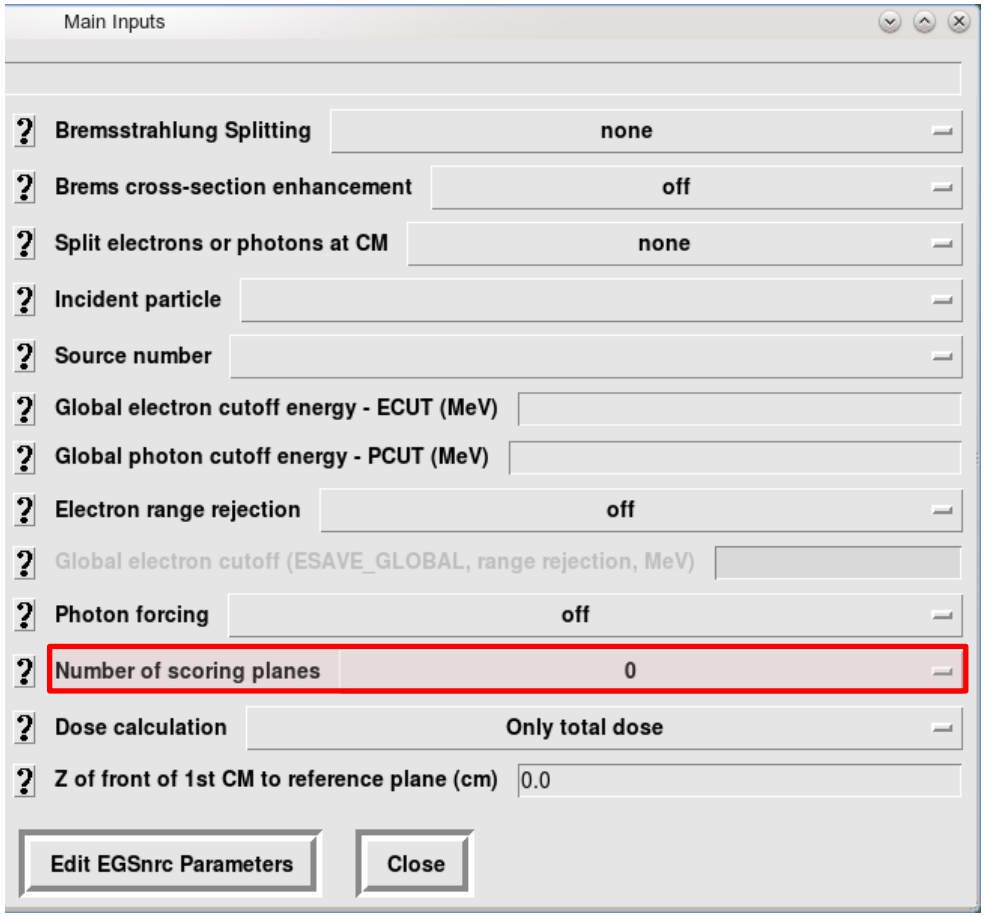


BEAMnrc main inputs: scoring and output

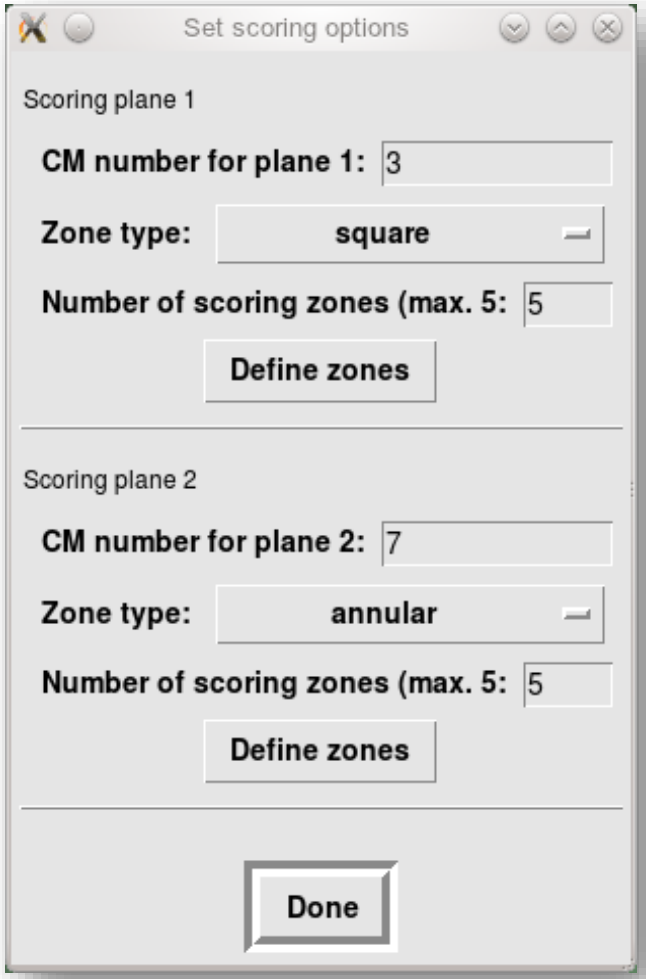


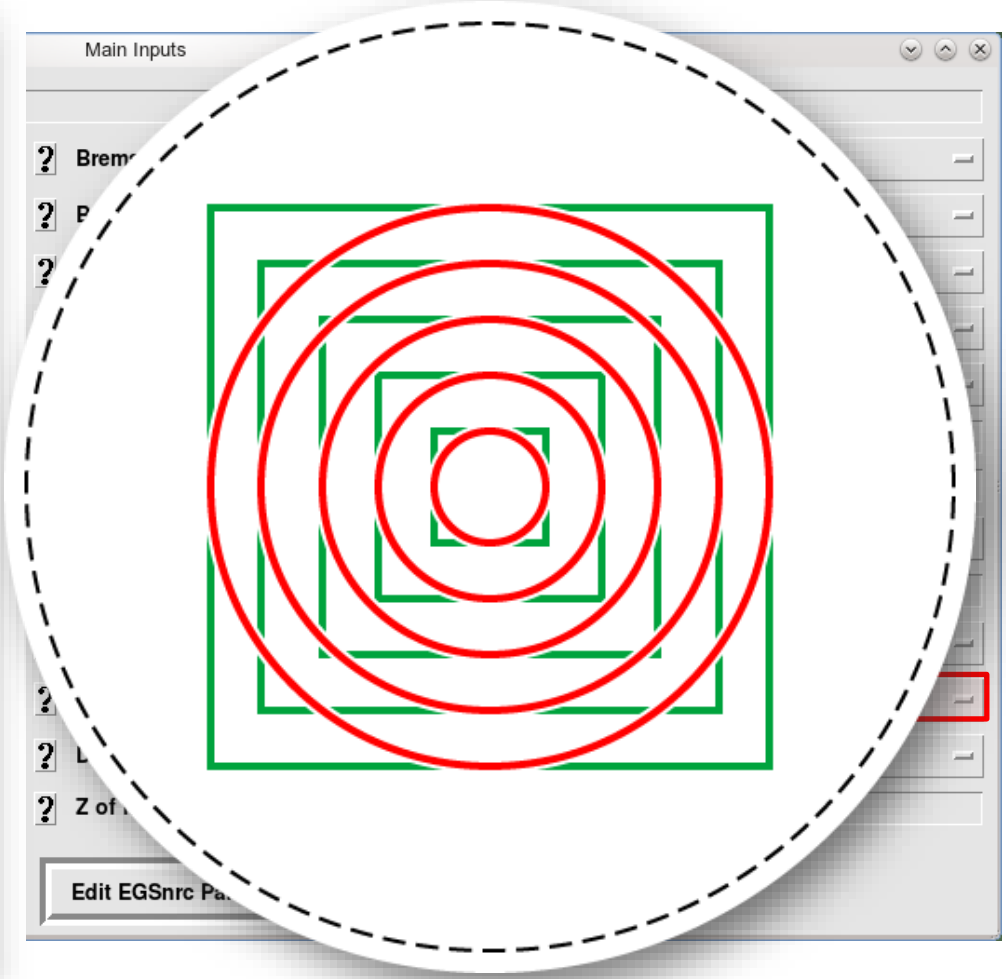
BEAMnrc main inputs: scoring planes



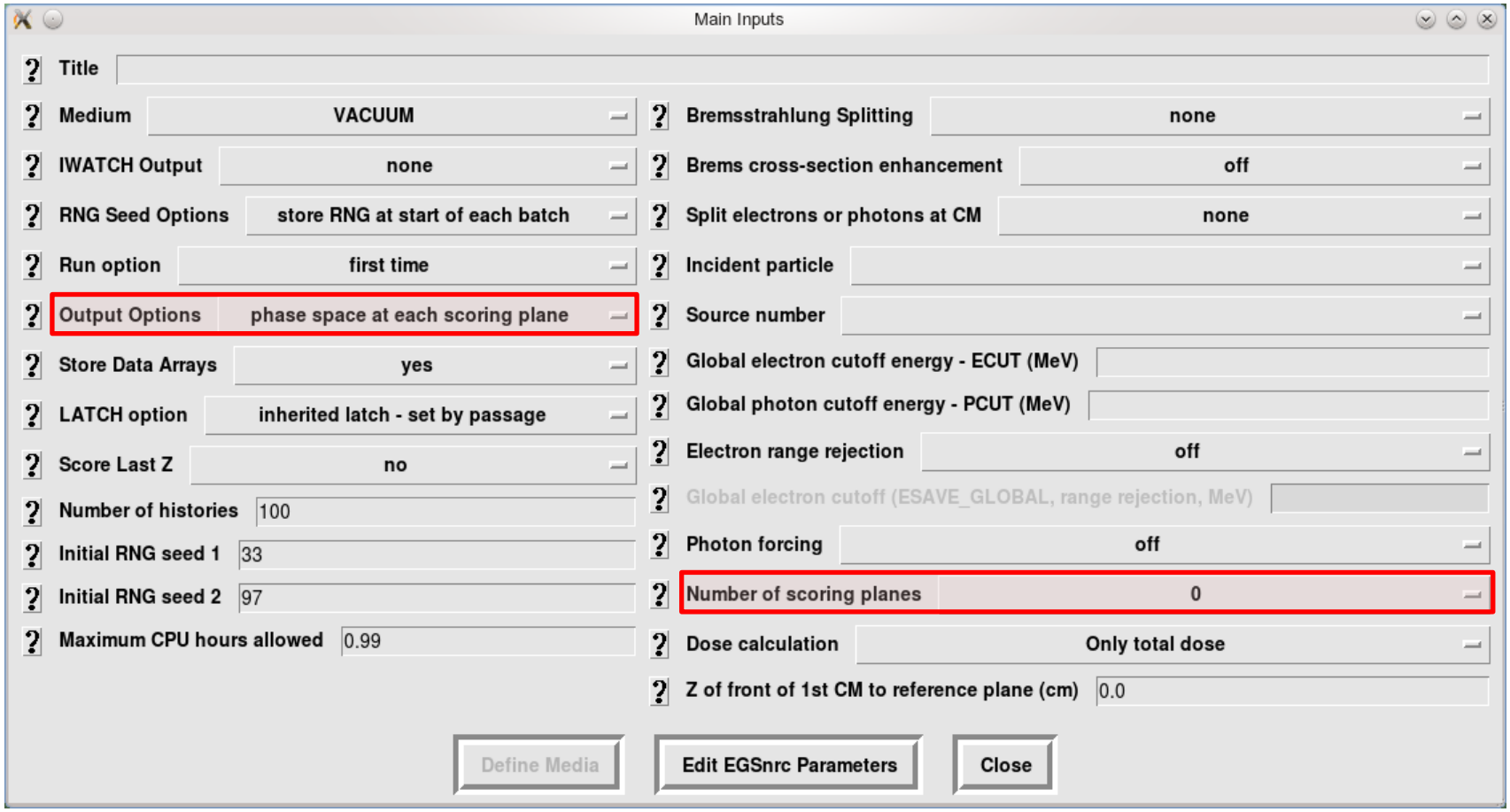


BEAMnrc main inputs: scoring planes

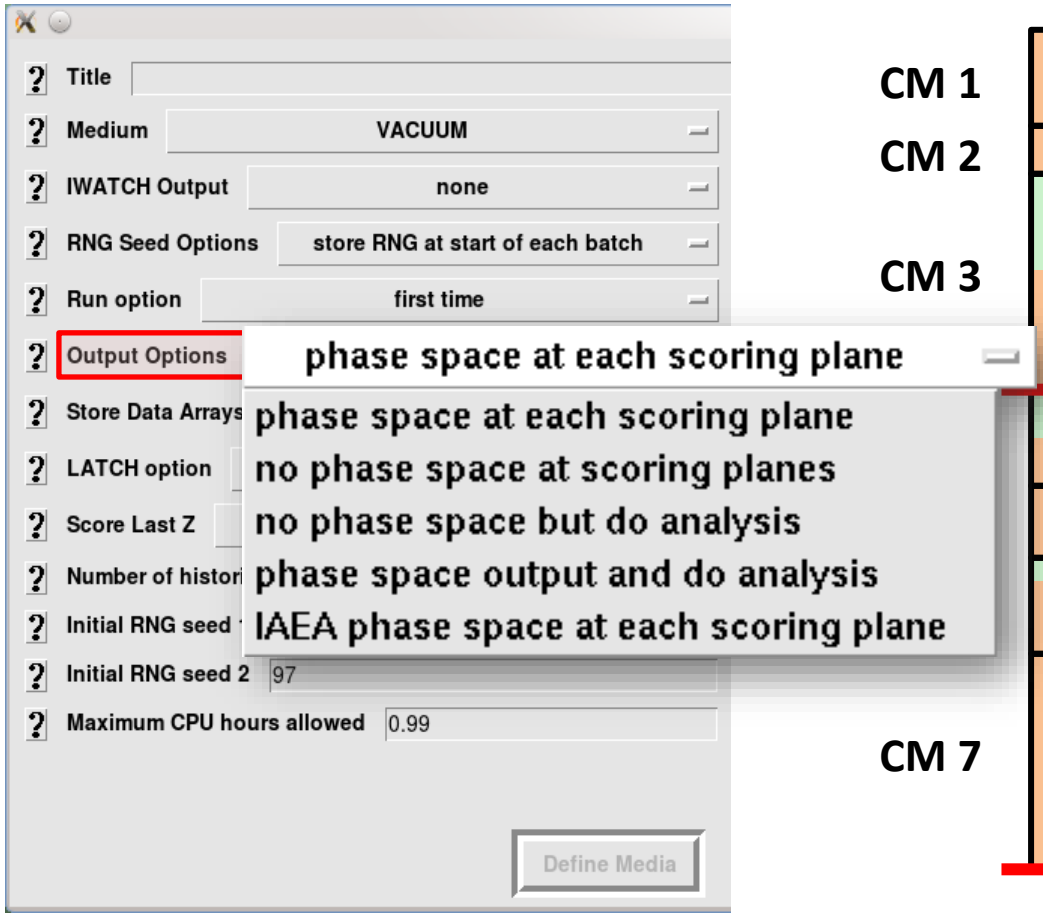


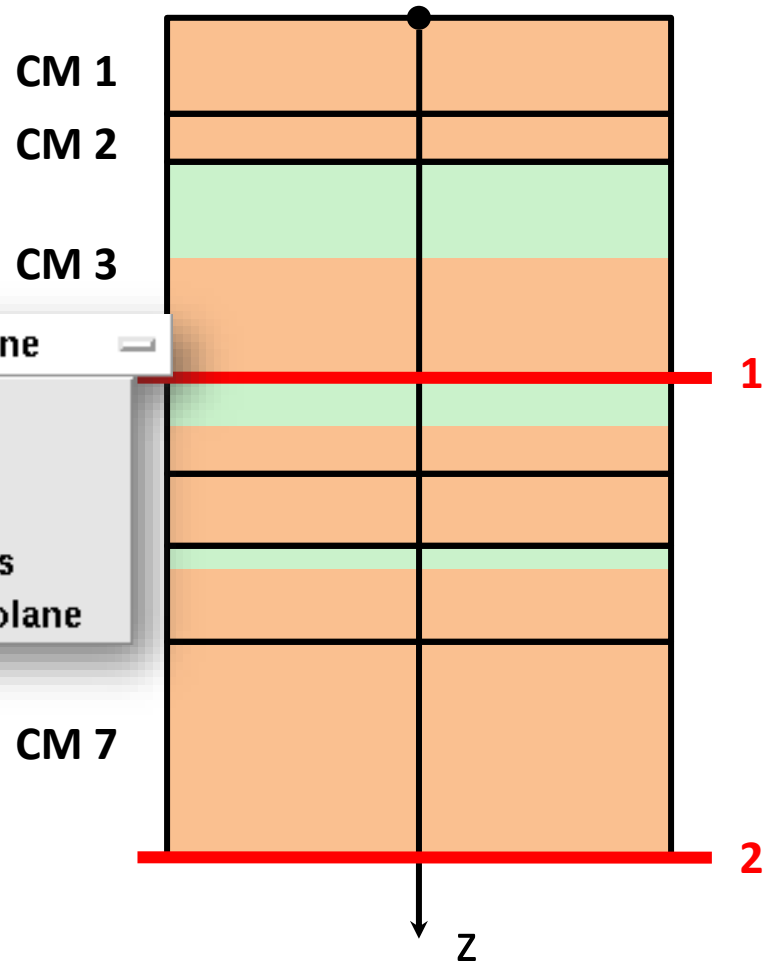


BEAMnrc main inputs: scoring planes



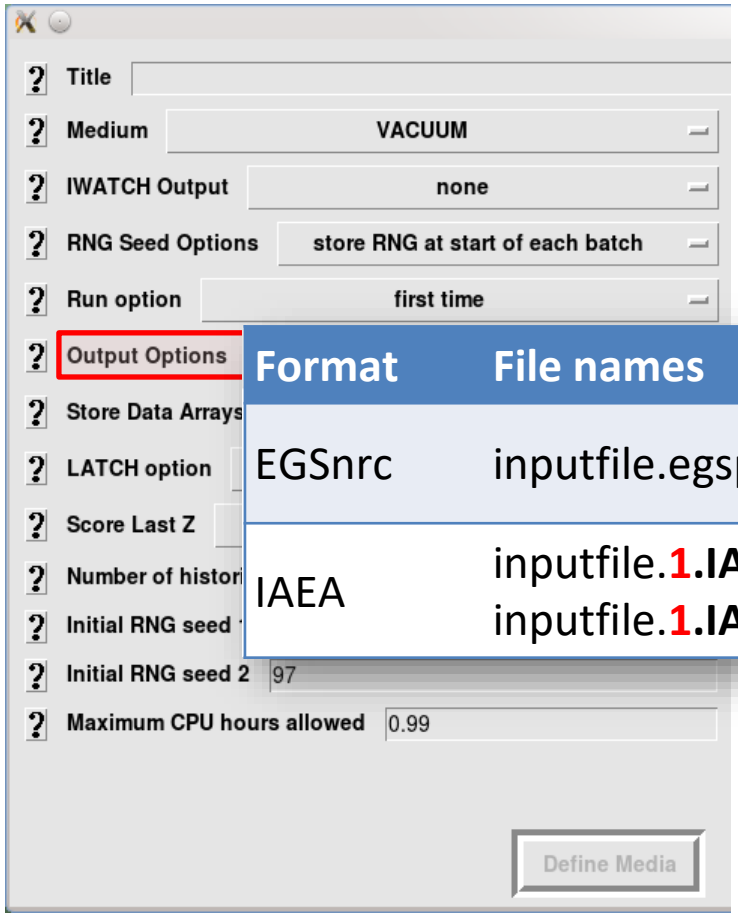
By default, phase space is stored at scoring planes



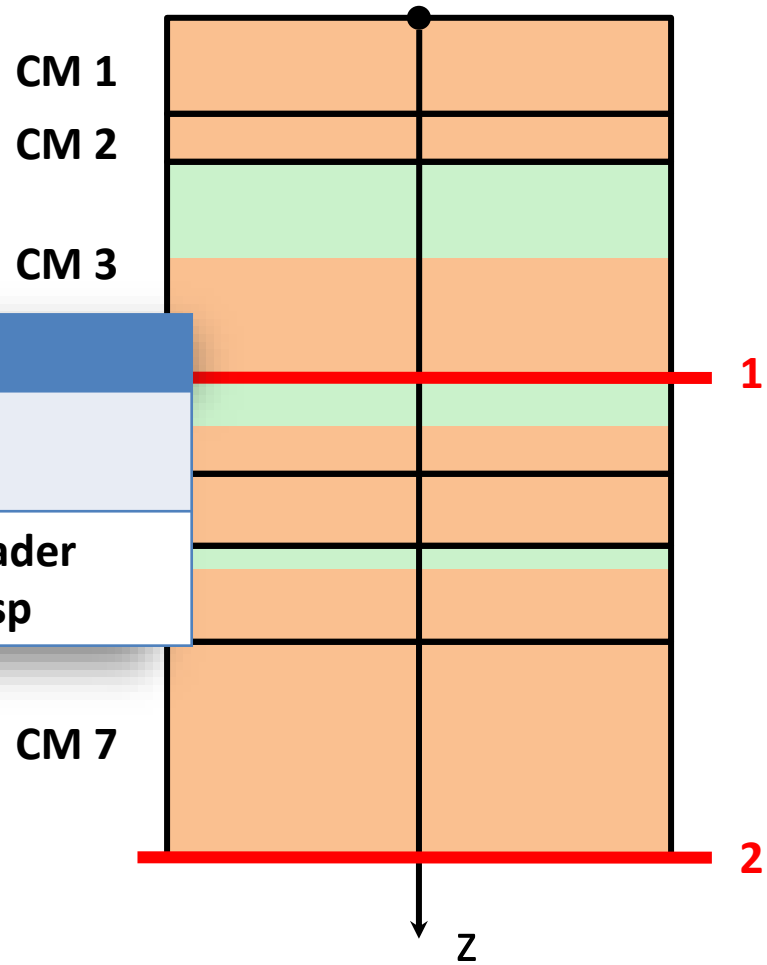


IO_OPT

By default, phase space is stored at scoring planes

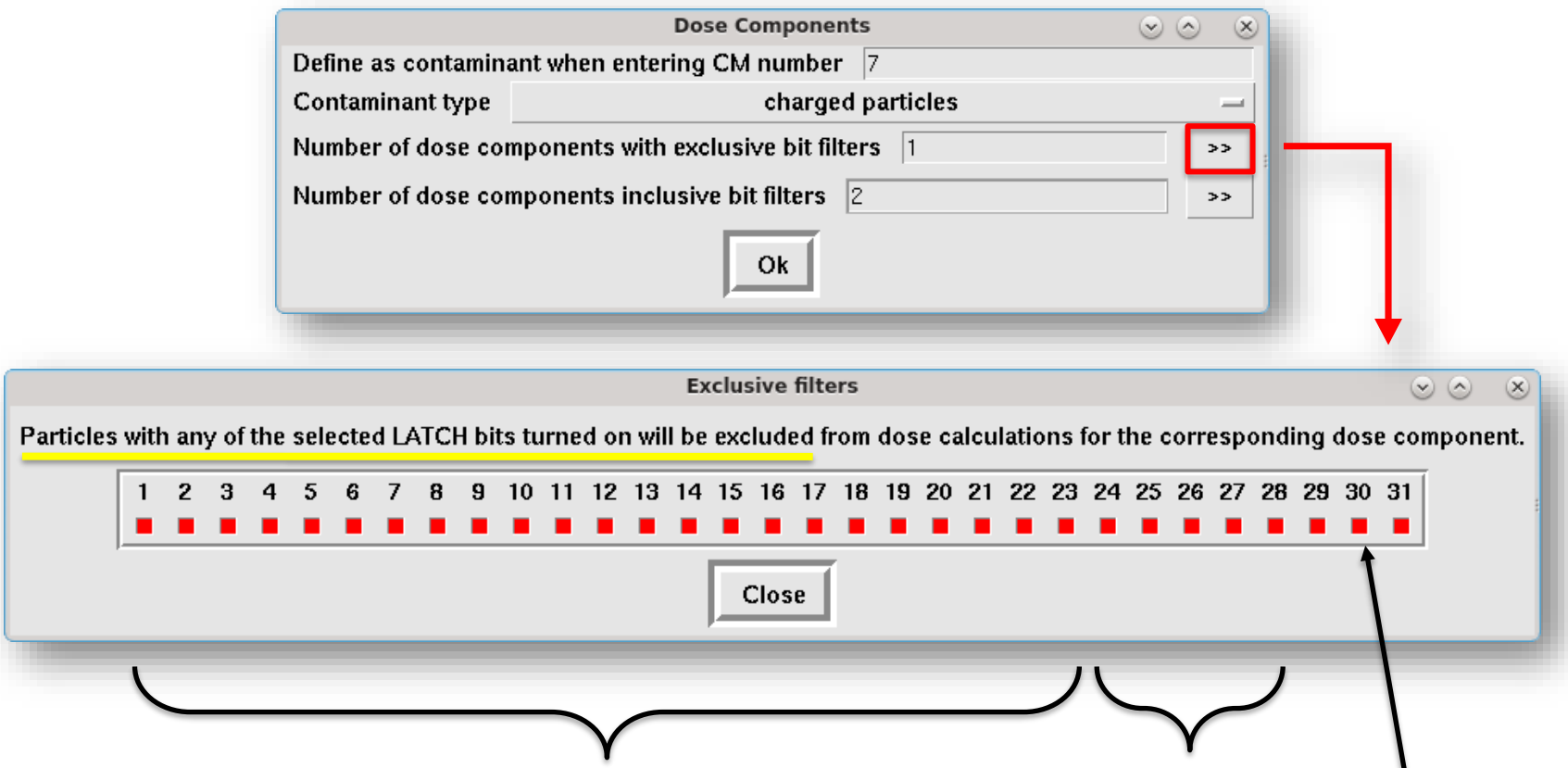


Format	File names
EGSnrc	inputfile.egsphsp**1**
IAEA	inputfile.**1**.**IAEAheader** inputfile.**1**.**IAEAphsp**



IO_OPT

LATCH bits are particle filters for dose components

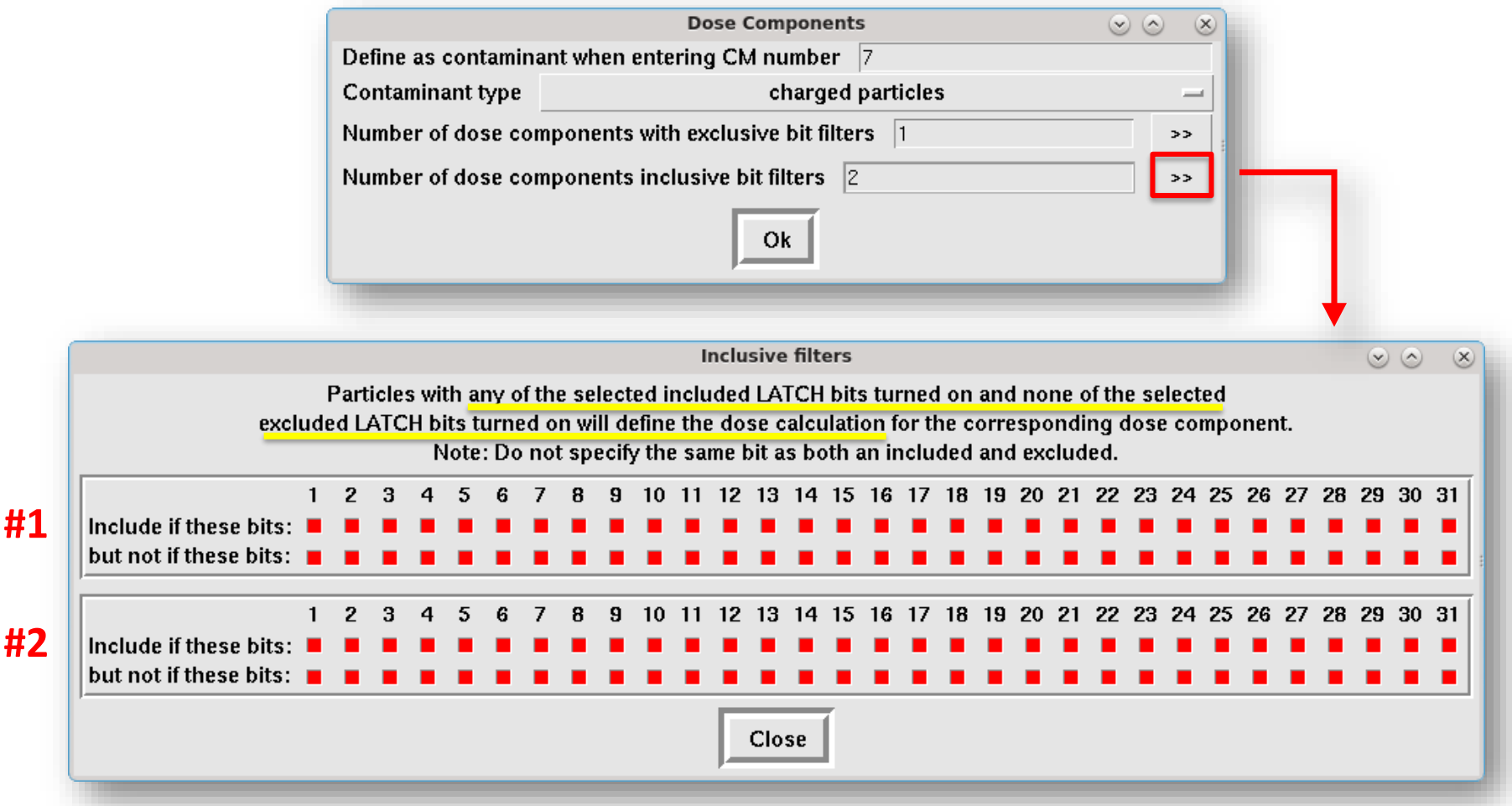


- bit region where particle **passed** or **interacted**
- when inherited, settings are "cumulative"

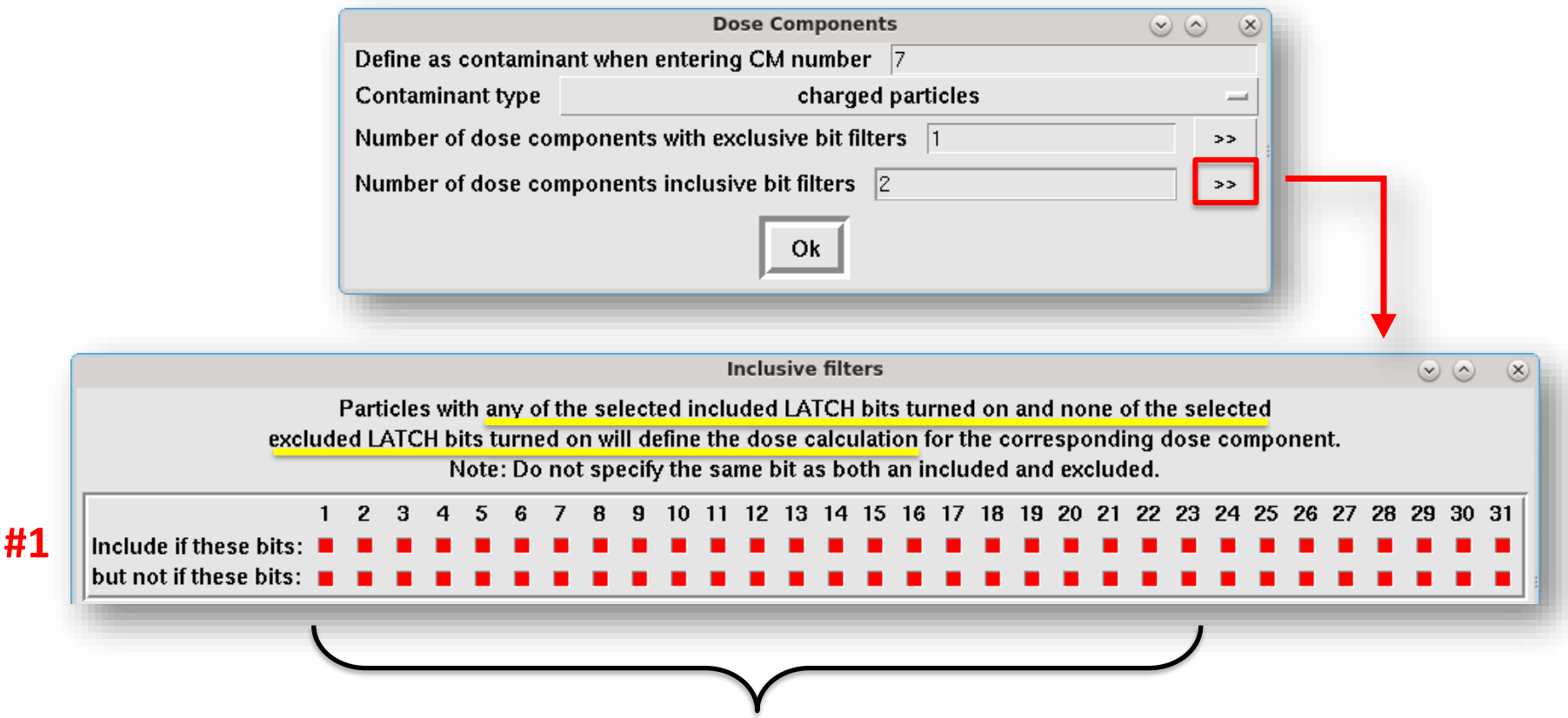
bit region number where secondary was created

contaminant

LATCH bits are particle filters for dose components



LATCH bits are particle filters for dose components



- bit regions are assigned during CM definition (see CM lecture)
- bits 1–23 can be assigned to regions
- **by default regions are assigned to bit 23**

Limits can be changed, but you have to recompile

- Defined in `$OMEGA_HOME/beamnrc/beamnrc_user_macros.mortran`
- **Copy** into `$EGS_HOME/BEAM_myaccel` directory to change for myaccel only
- Need to **recompile** (make) `myaccel` to apply changes

Macro	Description	Default
`$MAX_CMs`	maximum number of CMs	24
`$MXMED`	maximum number of media	**12**
`$MXREG`	maximum number of regions	3 000
`$MXSTACK`	maximum number of particles on stack	10 000
`$MAXBRSPLIT`	maximum brems splitting number	2 000
`$MAX_DOSE_ZONE`	maximum number of dose zones	1 500
`$MAX_SC_PLANES`	maximum number of scoring planes	**3**
`$MAX_SC_ZONES`	maximum number of scoring zones	**5**

Conclusions

- BEAMnrc is **installed by default** as part of EGSnrc.
- BEAMnrc is a **well-vetted workhorse EGSnrc application builder** for modelling "linear accelerators" (stacked components).
- **Recommendation:** use `beamnrc_gui` to edit your input files.
- BEAMnrc offers quite a complete library of linac **component modules.**
- Compile BEAMnrc as **a shared library** to use as a source (`make library`).
- BEAMnrc offers some **aggressive variance reduction techniques.**
- You can implement **your own component modules or sources,** based on existing ones (in MORTRAN).